



Connected and Automated Vehicles

Dr. Gokulnath

CART, IIT Delhi

- Why is the automotive industry changing now?
- What exactly are “connected” and “automated” vehicles?
- How did we get here historically?
- And what barriers remain?

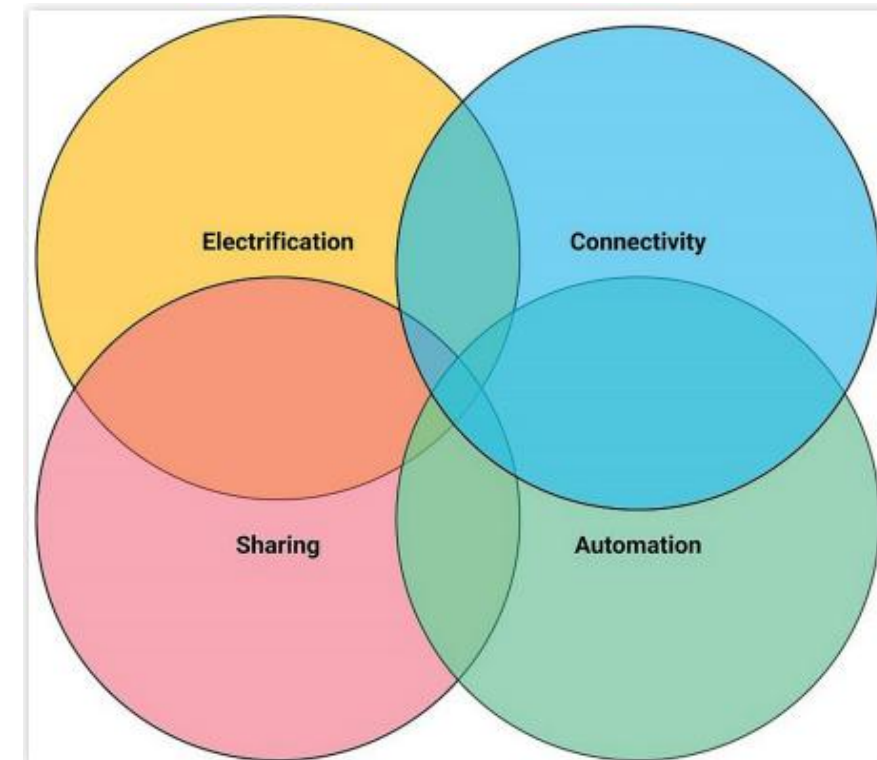


- Driver-centered paradigm stayed stable for ~100 years
 - Only incremental improvements
 - Human driver controls steering + pedals
 - Minimal communication (turn signals)
- Automotive industry
 - Improves safety, efficiency, onboard computing

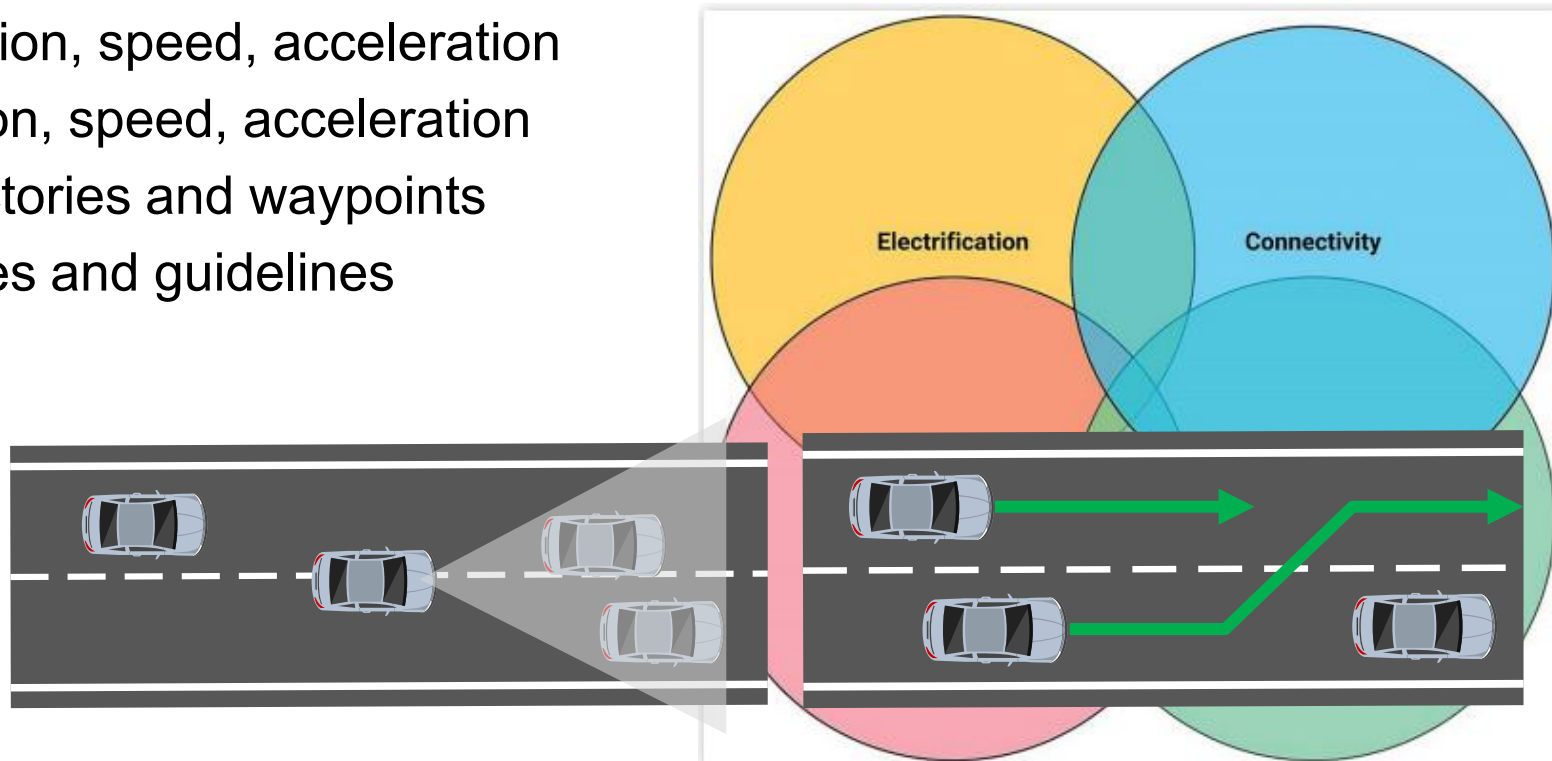
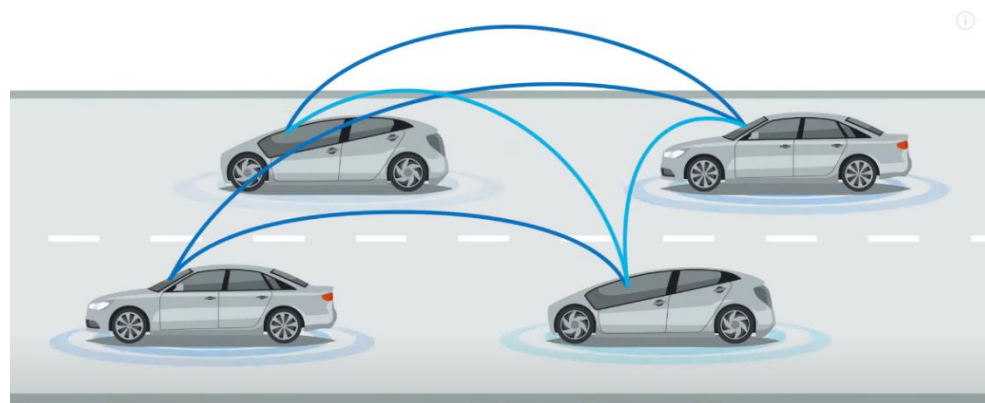


- Electrification

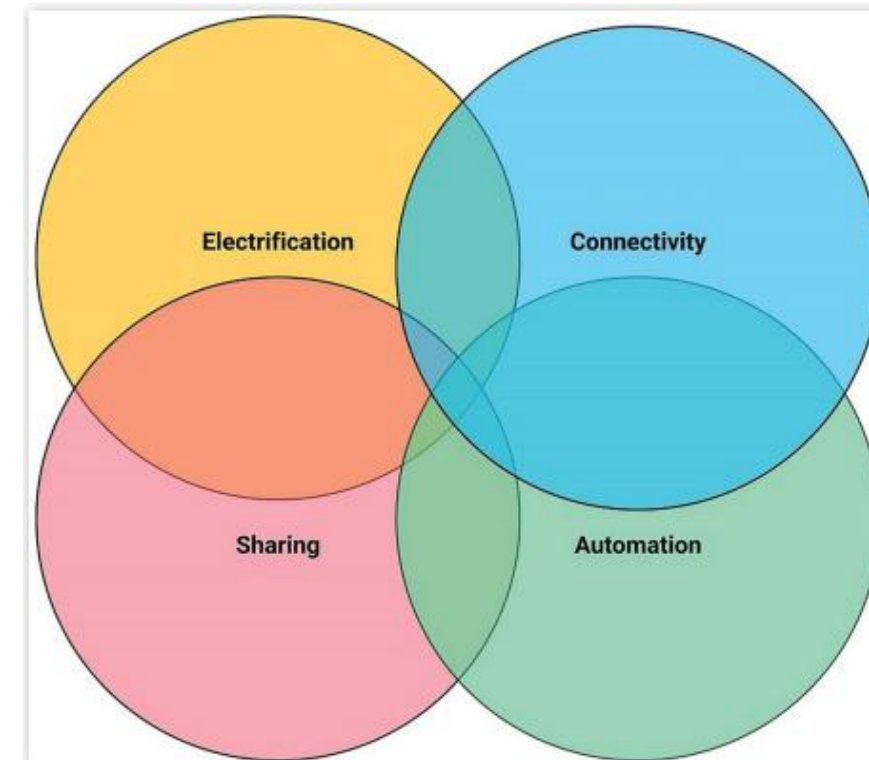
- Electrification is the most mature of the four trends
- Powertrain shift from Internal Combustion Engines (ICE) → hybrid/EV (electric motors)
- R&D work in vehicle electrification is only two decades old
- Norway, the percentage was 54% in 2020
- More efficient at converting energy to vehicle motion
- Fewer moving, mechanical parts → lower maintenance
- Drastically reduce emissions



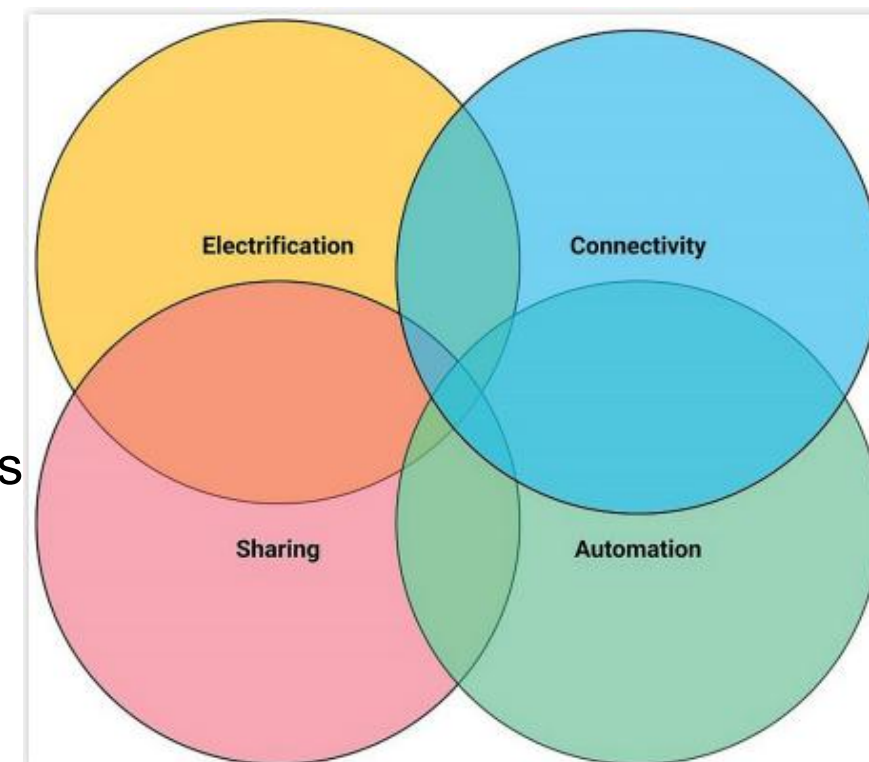
- Connectivity
 - Connectivity is the next mature trend
 - GPS-equipped vehicles were introduced by Oldsmobile Ciera 1994
 - Early vehicles can receive Signal Phase and Timing (SPaT) information from traffic signals
 - Provide information on the traffic light sequence and timing
- Possible information exchanged between vehicles
 - Own dynamic properties: position, speed, acceleration
 - Sensor detected object: position, speed, acceleration
 - Driving intentions: future trajectories and waypoints
 - Traffic management procedures and guidelines



- Sharing
 - Sharing is the second-least mature trend
 - Sharing includes ride hailing (Uber) and car sharing (AutoShare)
 - Sharing is also known as multi-modal transportation
 - From scooters to bicycles to cars to buses and trains
 - Sharing provide enhanced mobility for individuals including
 - disabled persons, seniors, children, etc.
 - Sharing allows for lower levels of vehicle ownership



- Automation
 - Automation = vehicles with **Automated Driving Systems (ADS)**
 - **Automation Spectrum (SAE J3016):**
 - Low Level: Driver Assistance (e.g., Adaptive Cruise Control)
 - High Level: Full Automation (No human supervision required)
 - Automation helps humans to focus on other tasks
 - Automation reduce accidents by removing human error
 - Automation helps for those who cannot drive themselves
 - Automation is the **biggest change** in the vehicle industry
 - Since the invention of the automobile itself
 - Automation is the "last in maturity" compared to other trends
 - Holds the highest potential for total sector disruption



- Airplane autopilot enabled flying + navigation simultaneously
 - Example: Sperry Gyroscope Autopilot (1930s)
 - Automatically maintain a desired compass heading and altitude
- Early torpedoes: maintain course + depth (1860s)
 - Later: added sonar targeting by WWII
- German V2 rocket: gyroscope-based guidance
 - Early human-made object into outer space



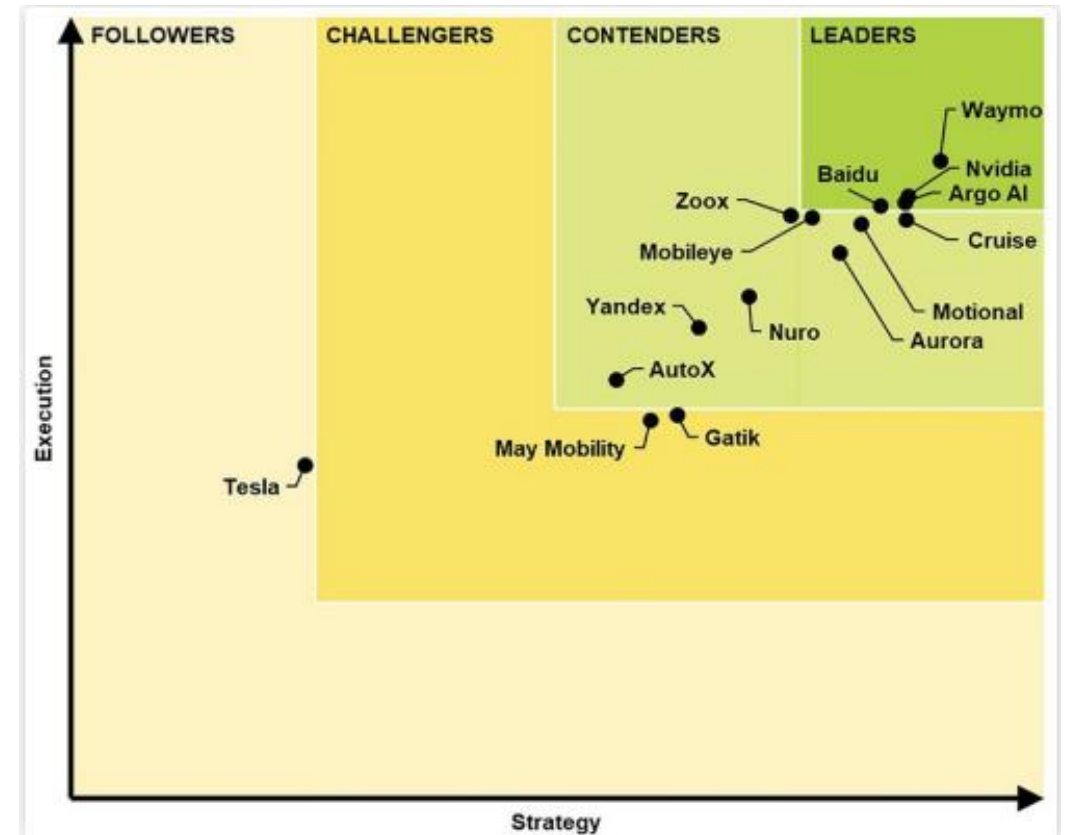
- Cockroach-like motion: Sensing → Processing → Reacting
 - Sensing and reacting were possible
 - Hardest part historically: **processing (machine intelligence)**
- 1980s–1990s: High-Speed Autonomy
 - The Mercedes Van (1980): Travel on highways using a primitive automated driving version
 - VaMoRs Van (1997): Third generation of advanced vision systems for autonomous navigation



- DARPA Challenge (2004–2007)
 - First long distance competition for driverless cars in the world
 - Stanley (2005) and Boss (2007) winners
 - Used machine learning and sensor fusion (LiDAR/Radar) for automation
 - Modern AV industry born
 - Creating companies like Waymo, Tesla, and Uber



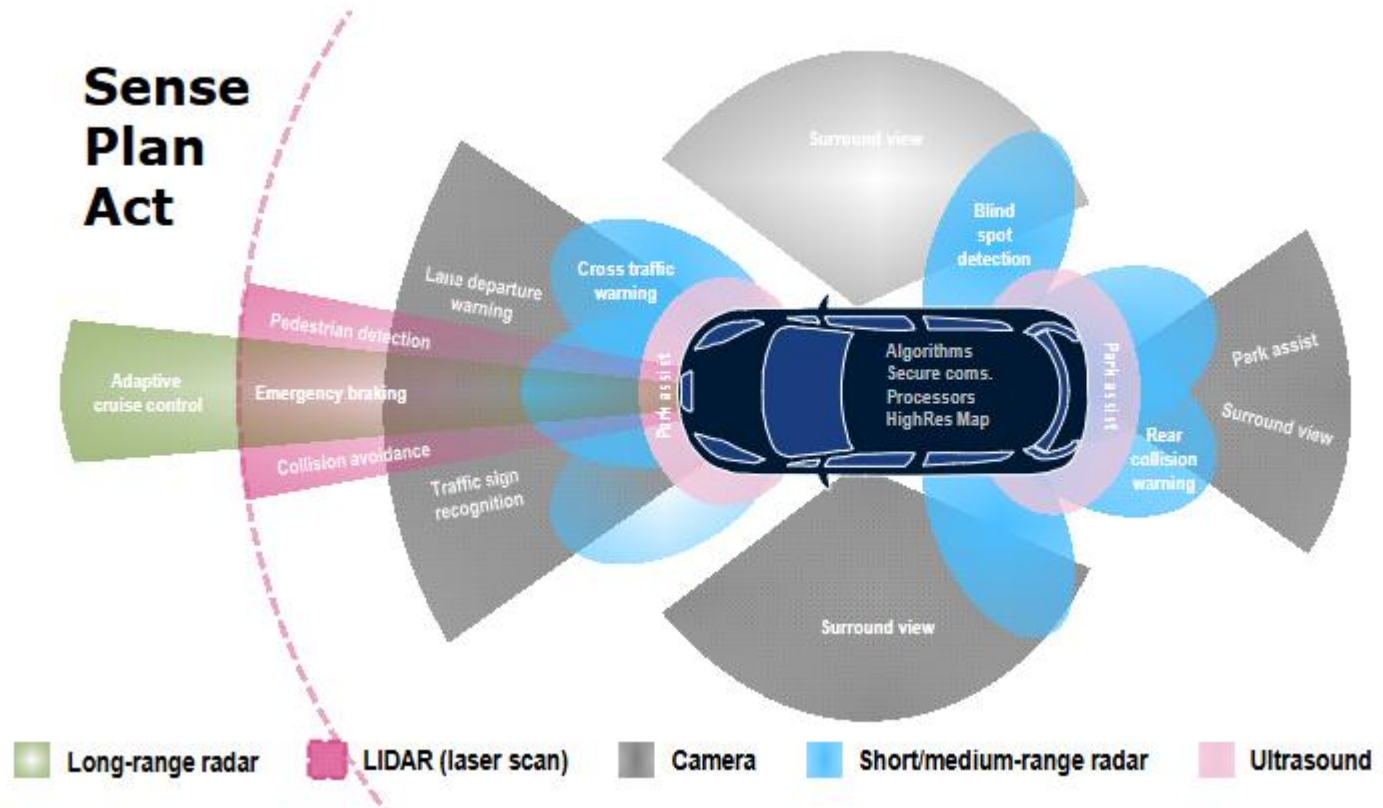
- CAV industry rankings criteria
 - Vision, Go-to market strategy, Partners, Production strategy, Technology, marketing, and distribution, Product capability, Product quality and reliability, Product portfolio, Staying power
- CAV industry leaders
 - Waymo, Nvidia, Argo AI, and Baidu
 - Tesla low in both strategy and execution
 - Public perception may differ



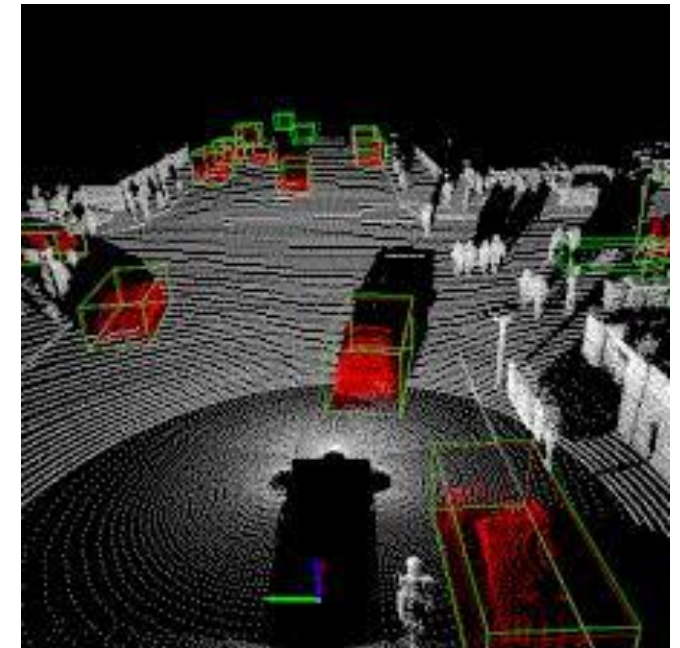
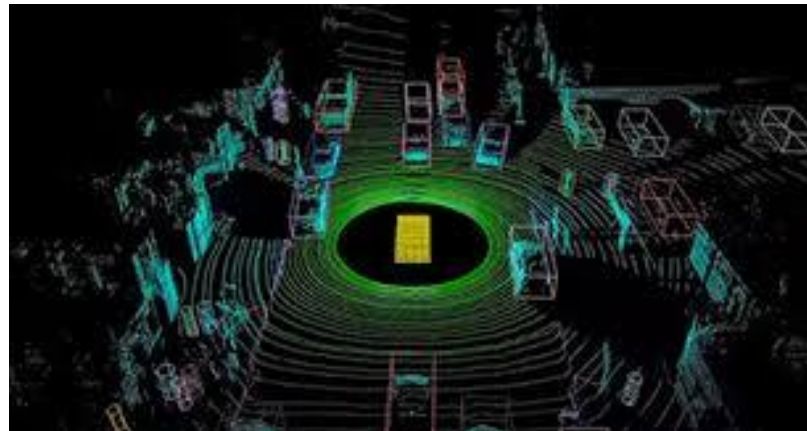
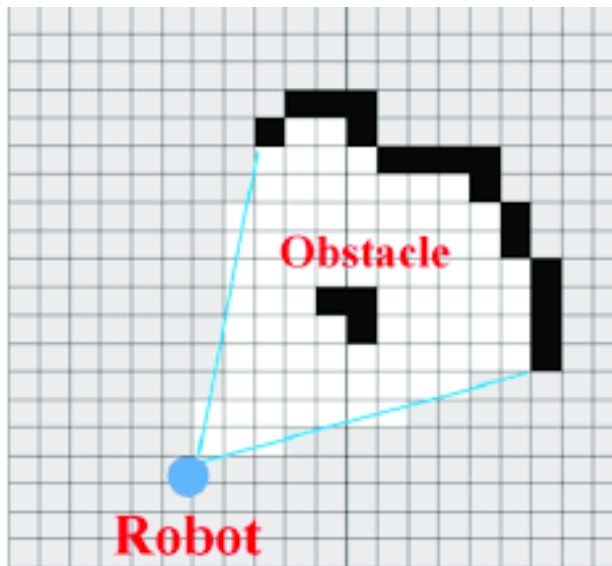
- Strong excitement from industry/government/public in 2020
 - But widespread deployment is not near-term
- **The 80/20 Rule:** The industry has mastered roughly 80–90% of driving tasks,
 - But the final 10–20% (corner cases) is proving exponentially more difficult
 - Perception must handle difficult scenarios:
 - Weather, temporary obstacles/restrictions, parking lots, heavy pedestrian/cyclist traffic, non-mapped areas
- **Timeline Debate:** Opinions are split
 - Some see mass commercialization by 2030, while others predict it is still decades away



- A CAV works
 - Perception: Understanding the world
 - Planning: Deciding what to do
 - Actuation/Control: Executing safely



- Perception must produce a machine-readable scene of the environment
 - Ego-state: position, velocity, yaw rate, acceleration (vehicle's own motion)
 - Objects: detection + classification (car, truck, pedestrian, cyclist...)
 - Tracking: position/velocity/direction over time of objects
 - Free space: drivable area, boundaries, curbs, road edges
 - Traffic controls: signals, signs, lane markings, right-of-way cues
- **Outputs:** occupancy grid, tracked objects list



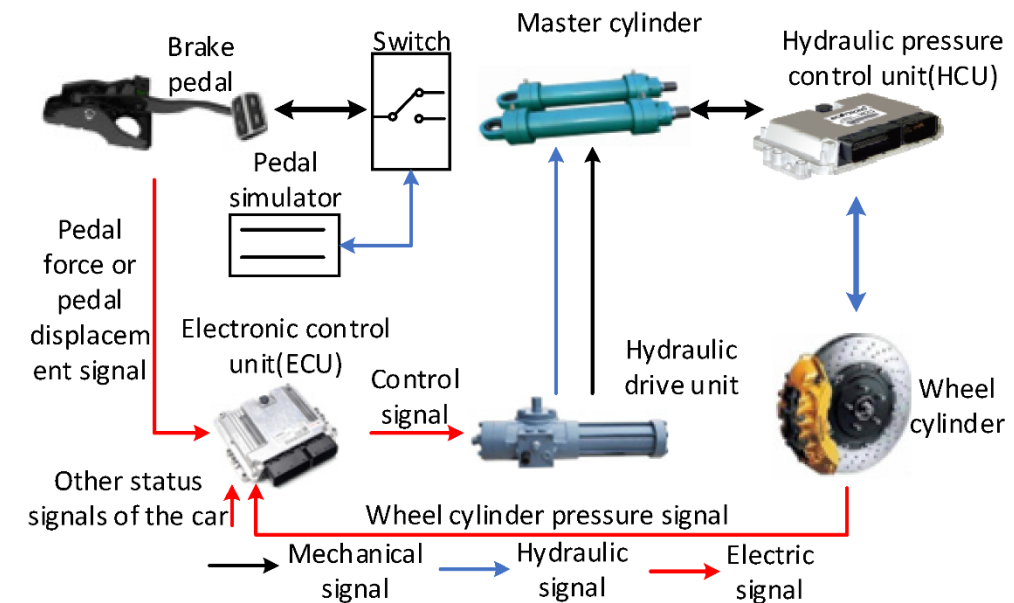
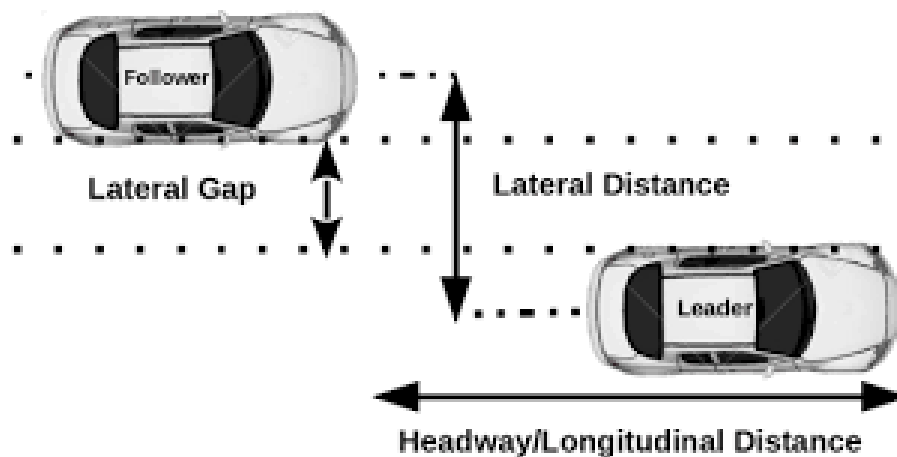
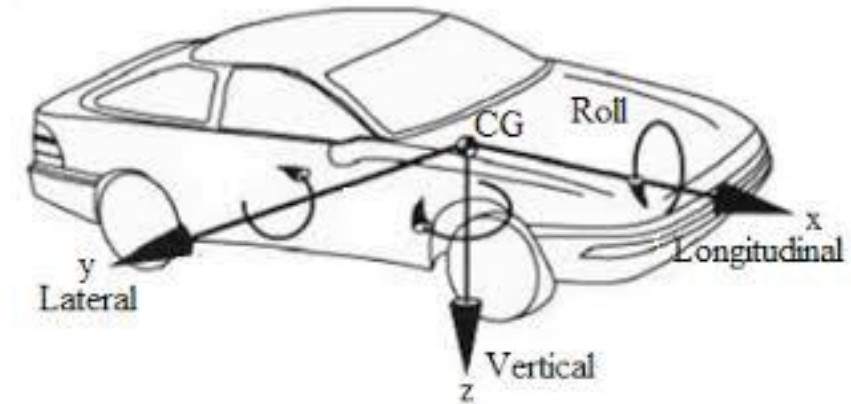
- Sensors (typical):
 - **Cameras:** rich semantics (lanes, signs), weak in glare/rain/night
 - **Radar:** strong range/velocity, weaker shape/class semantics
 - **LiDAR:** accurate geometry, cost/packaging concerns
 - **GNSS/IMU/Wheel odometry:** ego motion + localization backbone
- Fusion levels:
 - **Raw-data fusion** (hard, heavy compute)
 - **Feature-level fusion**
 - **Object-level fusion** (common, robust)



- Planning converts world model + mission into actions:
 - **Mission / Route planning:** where to go
 - **Behavior planning:** what maneuver (keep lane, yield, overtake, stop)
 - **Motion planning:** exact trajectory (path + speed profile) within constraints
- **Output:** a trajectory: $x(t), y(t), v(t)$ over next few seconds
- **Prediction**
 - Forecast trajectories of vehicles/pedestrians/cyclists
 - Estimate maneuver (will cut-in? will cross? will stop?)
- **Prediction Approaches:**
 - Physics-based (constant velocity/acceleration)
 - Rule-based (yielding, lane following)
 - Learning-based (data-driven intent + interaction)



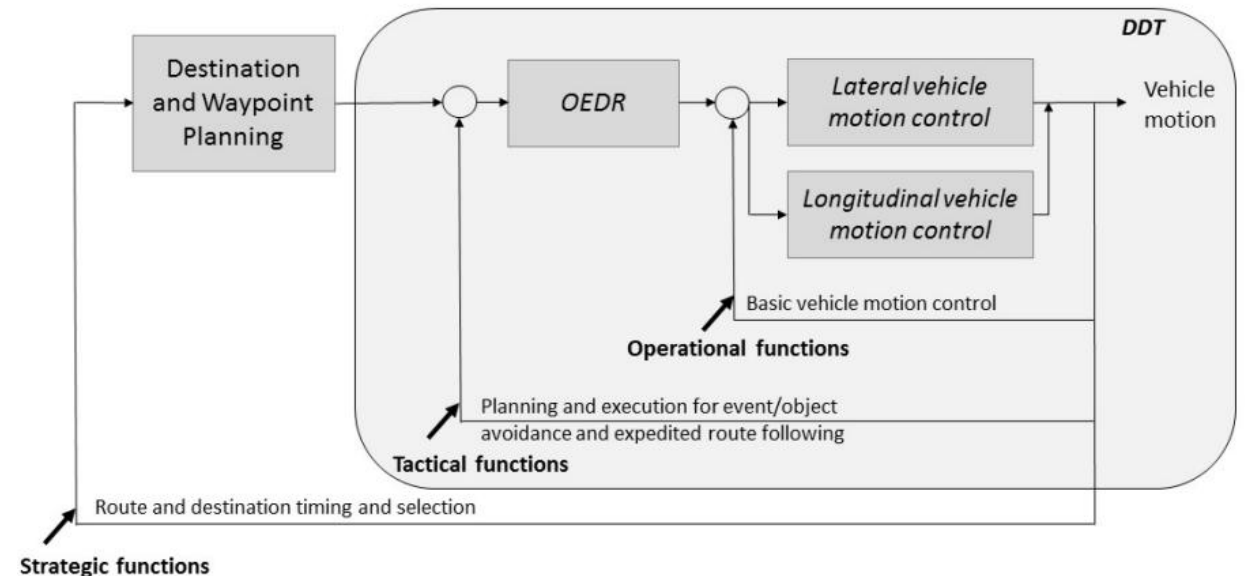
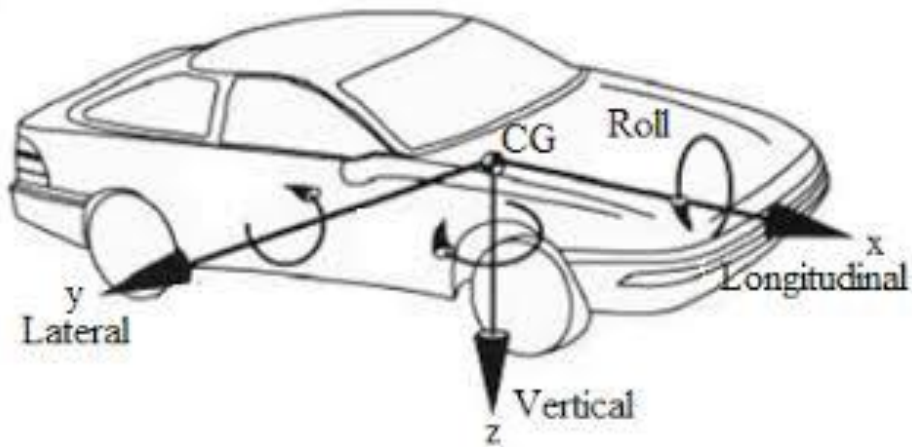
- Control turns planned trajectory into commands:
 - **Lateral control:** steering to follow path
 - **Longitudinal control:** throttle/brake to follow speed profile
- Vehicle actuators:
 - Steering actuator, brake-by-wire, throttle, gear, etc.
 - Safety monitoring: actuator faults, degraded modes
- **Output:** steering angle + throttle/brake commands at high rate



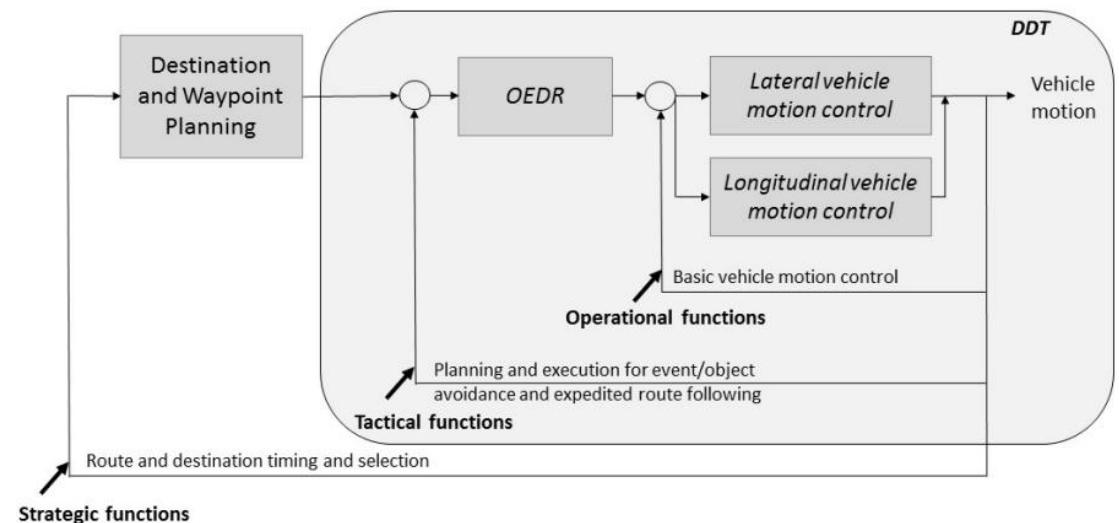
- What is SAE J3016?
 - Defines a **taxonomy** with **six levels of driving automation (0–5)** for on-road vehicles
 - Levels apply to **features**, not necessarily the whole vehicle
 - Car might have a Level 2 highway feature and a Level 4 parking feature
- Three actors model involved
 - **The (Human) User**: Can be a driver, passenger, or remote assistant.
 - **The Driving Automation System**: The hardware/software performing the task.
 - **Other Vehicle Systems**: Brakes, steering, lights (the "conventional" parts of the car)
- Active Safety is **not** driving automation because it is momentary, not sustained
 - Anti-lock Braking System (**ABS**), Electronic Stability Control (**ESC**), Automatic Emergency Braking (**AEB**)



- Dynamic Driving Task (**DDT**)
 - System performs **real-time operational + tactical** driving functions, excluding **strategic** trip planning
- **Strategic**: Trip planning (Destination, Waypoints)
 - This is outside the scope of DDT
- **Tactical**: Maneuver planning (changing lanes, signaling)
- **Operational**: Split-second micro-adjustments (steering, braking)



- **ODD (Operational Design Domain)**
 - Set of operating conditions under which the driving automation feature is designed to function
 - e.g., road type, speed range, geography, weather/visibility, traffic conditions, time-of-day
- **OEDR (Object and Event Detection and Response)**
 - Monitoring the driving environment (detect/recognize/classify objects and events)
 - Responding appropriately to complete the DDT and/or support fallback
- **Key:**
 - ODD = What conditions is this feature supposed to operate
 - OEDR = What it must perceive and how it responds within those conditions



- DDT fallback triggered by either
 - **DDT performance-relevant system failure**
 - Malfunction that prevents the automation system to perform DDT
 - Eg: sensor outage (camera/radar), steering/brake actuator fault, compute failure
 - **ODD (Operational Design Domain) exit**
 - Vehicle/feature is no longer operating within ODD
 - Eg: road type changes (highway → city), speed out of range, heavy fog/rain, construction zone
- DDT fallback outcome
 - **Driver takeover:** Human performs the DDT (lateral + longitudinal control)
 - **Minimal Risk Maneuver:** Automated Driving System (ADS) move to a safe minimal-risk state
 - Move to a safe location and stop
 - Reduces crash risk

- **Definition:** Driver performs the entire **Dynamic Driving Task (DDT)** at all times
- **Key points:**
 - Warnings / momentary interventions do **not** count as automation (not sustained DDT control)
 - Driver is always responsible for steering, speed control, and environment monitoring
- **Examples:** Anti-lock Braking System (ABS), Electronic Stability Control (ESC), Automatic Emergency Braking (AEB), Forward Collision Warning (FCW), Lane Departure Warning (LDW)
- **Roles at Level 0:**
 - Dynamic Driving Task (**DDT**): Driver
 - Object and Event Detection and Response (**OEDR**): Driver
 - **DDT** fallback: Driver
 - Operational Design Domain (**ODD**): Not applicable (no driving automation feature)



- **Definition:** System performs sustained **either lateral OR longitudinal** motion control (not both)
- **Key points:**
 - Driver performs OEDR (continuous supervision and environment monitoring)
 - Driver controls the other axis not controlled by the system
- **Examples:**
 - Adaptive Cruise Control (ACC): controls speed + gap (longitudinal only); driver steers
 - Cruise Control (CC): controls speed (longitudinal only); driver steers
 - Lane Keeping Assist (LKA): controls steering (lateral only); driver controls speed/brake
- **Roles at Level 1:**
 - Dynamic Driving Task (DDT): Driver + System (system controls one axis)
 - Object and Event Detection and Response (OEDR): Driver
 - DDT fallback: Driver
 - Operational Design Domain (ODD): Feature-specific



- **Definition:** System performs sustained **both** lateral **and** longitudinal motion control simultaneous.
- **Key points:**
 - Driver performs OEDR and must supervise continuously.
 - Driver must be ready to intervene immediately at any time.
- **Examples:**
 - Highway Assist / Pilot Assist: controls steering + speed at the same time; driver still monitors
 - Traffic Jam Assist: low-speed stop-and-go and lane centering simultaneously; driver supervises.
 - Traffic-Aware Cruise Control: steering + speed control; driver performs OEDR and fallback.
- **Roles at Level 2:**
 - Dynamic Driving Task (DDT): Driver + System (system controls lateral + longitudinal)
 - Object and Event Detection and Response (OEDR): Driver
 - DDT fallback: Driver
 - Operational Design Domain (ODD): Feature-specific



- **Definition:**
 - **Automated Driving System (ADS)** performs the entire **DDT including OEDR** within its **ODD**; human is **fallback-ready** and must respond to a **request to intervene**
- **Key points:**
 - User is not required to supervise continuously while ADS is engaged
 - When ADS requests takeover, the user performs fallback (takes over DDT)
- **Examples:** Traffic jam ADS within defined conditions (limited-access roads, speed range, etc.)
- **Roles at Level 3:**
 - Dynamic Driving Task (DDT): ADS
 - Object and Event Detection and Response (OEDR): ADS
 - DDT fallback: Fallback-ready user (upon request)
 - Operational Design Domain (ODD): Required and must be stated



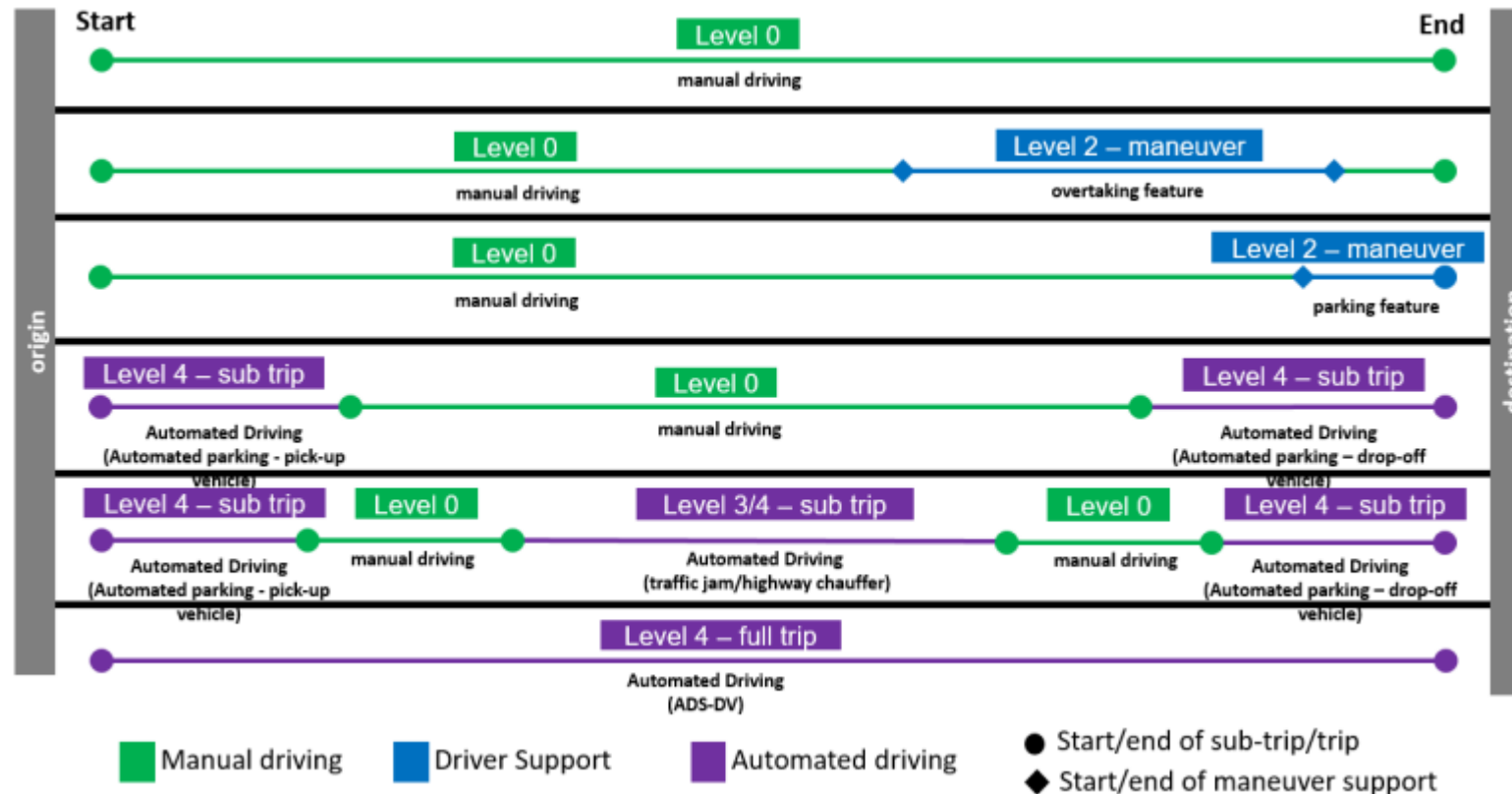
- **Definition:** ADS performs **DDT + OEDR + DDT fallback** within its **ODD** (no expectation of human takeover).
- **Key points:**
 - If a failure occurs or ODD is reached/exceeded, ADS handles fallback (e.g., achieves minimal risk condition).
 - Operation is limited to its ODD (geo-fenced area, specific roads, conditions, etc.).
- **Examples:**
 - Geo-fenced robotaxi, automated shuttle on fixed routes, driverless valet parking in controlled areas
- **Roles at Level 4:**
 - Dynamic Driving Task (DDT): ADS
 - Object and Event Detection and Response (OEDR): ADS
 - DDT fallback: ADS
 - Operational Design Domain (ODD): Required and must be stated



- **Definition:** ADS performs **DDT + OEDR + fallback** under **all conditions a human driver could manage**.
- **Key points:**
 - No operational limitation by ODD (conceptually “anywhere a human can drive”).
 - No human driver needed at any time.
- **Examples:**
 - Concept level; no broadly deployed unrestricted Level 5 systems today
- **Roles at Level 5:**
 - Dynamic Driving Task (DDT): ADS
 - Object and Event Detection and Response (OEDR): ADS
 - DDT fallback: ADS
 - Operational Design Domain (ODD): Not limited (no defined constraints)



- Sub-trip feature:
 - Requires a human driver for at least part of every trip
- Full-trip feature:
 - Automated Driving System (ADS) features that can operate throughout complete trips



- Step 1: Sustained automation of the Dynamic Driving Task (DDT)?
 - Ask: Does the system perform any part of the DDT on a sustained basis (not just warnings or brief interventions)?
 - **If No → Level 0**
- Step 2: How many motion-control axes are automated (when engaged)?
 - One axis only (either lateral steering OR longitudinal accel/brake) → Level 1
 - **Both axes simultaneously (lateral + longitudinal together) → Level 2**
- Step 3: Who performs Object and Event Detection and Response (OEDR)?
 - If the driver must continuously monitor the environment and respond → Level 1 / Level 2
 - **If the system performs OEDR as part of driving → it is an Automated Driving System (ADS) → Level 3+**



- **Step 4: Who performs DDT fallback when needed (system failure or ODD exit)?**
 - If the feature expects a **human takeover** after a request to intervene → **Level 3**
 - If the **ADS performs fallback** and can reach a **minimal risk condition** without expecting human takeover → **Level 4 / Level 5**
- **Step 5: Is the ADS limited to a defined Operational Design Domain (ODD)?**
 - **Yes (ODD-limited: specific roads/area/speeds/weather conditions, etc.)** → **Level 4**
 - **No ODD limitation** (can drive under all conditions a human driver could manage) → **Level 5**
- **Important:**
 - Assign levels using **design intent + specified user role + stated ODD**, not “what it looked like” in a demo.



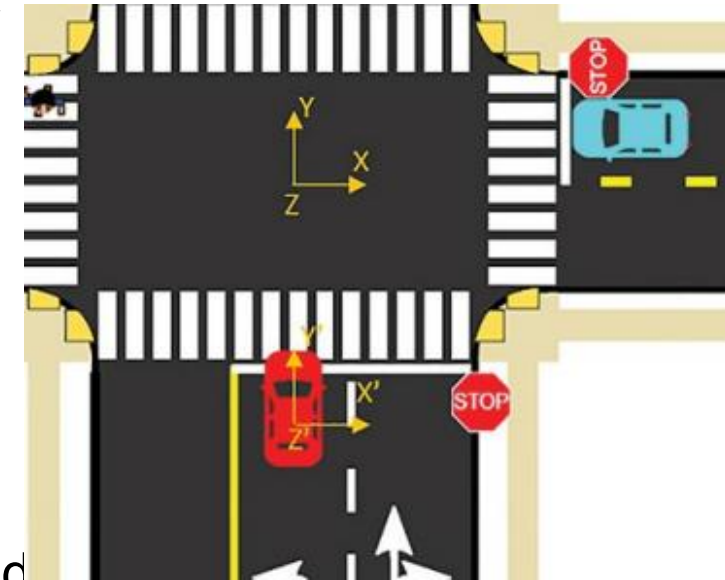
- ACC — Adaptive cruise control
- ADAS — Advanced driver assistance system
- ADS — Automated driving system
- ADS-DV — Automated driving system-dedicated vehicle
- AEB — Automatic emergency braking
- DDT — Dynamic driving task
- ESC — Electronic stability control
- LKA — Lane keeping assistance
- ODD — Operational design domain
- OEDR — Object and event detection and response



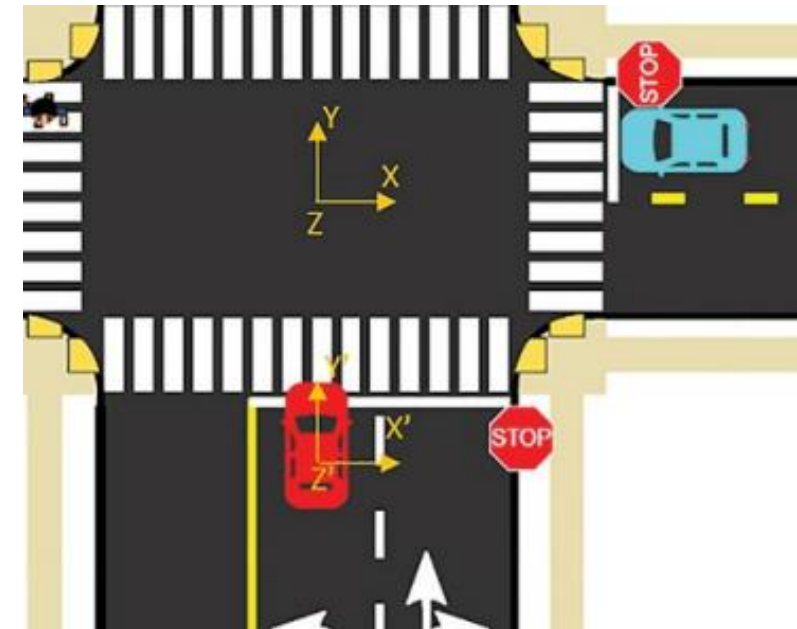
- **Definition:** Localization is the process by which a CAV determines its pose
 - 3D orientation, 3D position, speed, and acceleration
- **Core Question:** It answers the question?
 - Where am I exactly, and how am I moving relative to the known world?
- **Context:** It is the first step in the **Perception sub-system**.
 - Without knowing where it is, the vehicle cannot plan a path



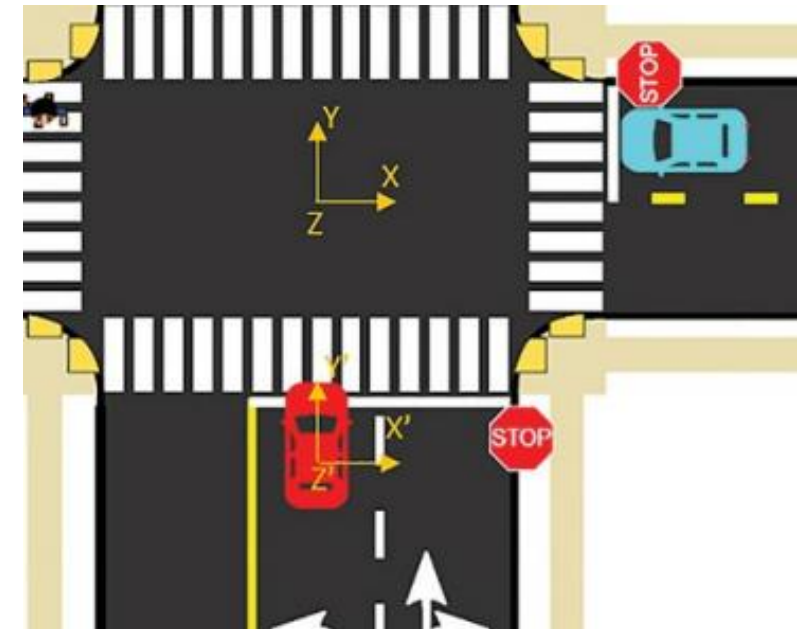
- **World Coordinate System** (X, Y, Z)
 - An arbitrary origin point (X, Y, Z) fixed on the map (e.g., the center of an intersection)
 - The choice of origin is arbitrary and determined by the developer
- **Local Coordinate System** (X', Y', Z')
 - The vehicle's own internal axes
 - Described relative to the World Coordinate System
- Localization algorithms calculate
 - Relationship between the vehicle's local frame and the fixed world frame
 - Vehicle's position (distance along axes) and orientation (Pitch, Roll, Yaw)



- **Sensors Used:** Cameras, Inertial Measurement Unit (IMU)
- Case Study: Vehicle Stopped Straight in Lane
 - Yaw (Rotation about Z'): 0° (Vehicle is straight)
 - Roll (Rotation about Y'): 0° (Assuming road is flat)
 - Pitch (Rotation about X'): 0° (Assuming road is flat)
 - **Note:** In this alignment, the pose is identical with respect to both local and world systems



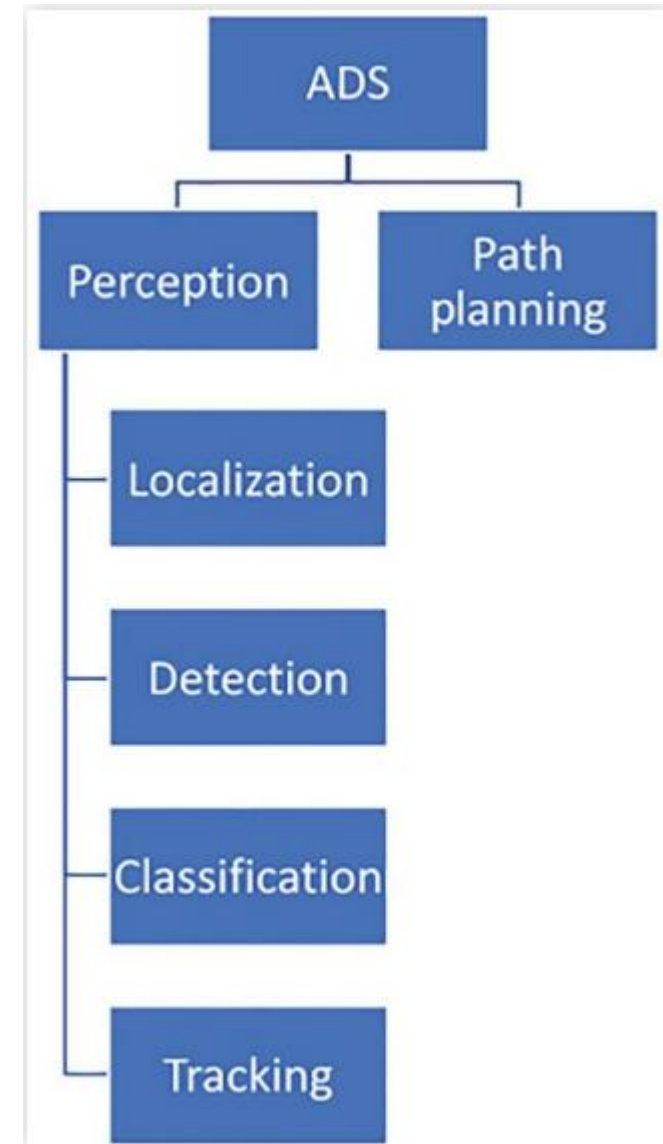
- **Sensors Used:** GPS Unit
- Case Study: Vehicle Stopped Straight in Lane
 - **X-Axis:** Distance is zero (assuming Y and Y' axes align)
 - **Z-Axis:** Distance is zero (assuming flat road/same elevation)
 - **Y-Axis:** Distance is a positive, nonzero value
 - Representing the distance from the intersection center to the vehicle



- **Sensors Used:** Wheel Speed Sensors
- Case Study: Vehicle Stopped Straight in Lane
 - Speed and acceleration are zero
- Case Study: Turning Left (Dynamic)
 - Pitch, Roll, and Yaw values change due to steering input and suspension movement



- Localization is considered a part of the perception system
 - Input to the path planning system
- **Redundancy:**
 - Localization acts as a perception redundancy check
 - It compares what the sensors "see" against
 - what the map says *should* be there
- **Ground Truth:**
 - A highly accurate map establishes "Ground Truth."
 - If sensors fail (blind spots)
 - The map provides safety-critical location data for static objects
- Localization consists of two primary steps:
 - **Mapping and Sensing**



- Mapping
 - Mapping is a fundamental pillar of localization.
- CAVs obtain environmental maps through three primary methods
 - **Connectivity**: Received from an outside source (V2X or Cloud)
 - **On-board Storage**: Pre-loaded and stored on the vehicle's computer
 - **Real-time Creation**: Generated dynamically by the vehicle's sensors as it moves
- Current Industry Standard
 - Most ADS developers rely on **a priori** mapping (pre-existing maps) within **geo-fenced** areas
 - In geo-fenced operations, the vehicle accesses the map through:
 - Connectivity, or On-board stored maps
 - **Real-time mapping alone** is generally considered **insufficient** by most ADS developers
 - **Tesla** is a major exception (discussed later in the chapter)



- Conventional (SD) Maps
 - SD map providers: **Google Maps, Apple Maps, Waze**, etc
 - SD maps are useful for **human navigation** but **insufficient** for CAV localization
 - Missing lane widths, curb heights, and specific signage data
 - SD map **positional accuracy** is often **off by meters**, which is unacceptable for CAVs
 - Study reported **Google Earth** accuracy ranging from **0.4 m to 171.6 m**
- 3D High-Definition (HD) Maps
 - Provide centimeter-level resolution/accuracy
 - Establishes the "**Ground Truth**" for the vehicle
 - If sensors fail or have blind spots, the HD map provides the location of static objects



- **Reduced Computational Load**
 - Highly detailed maps allow the ADS to focus processing power on **change detection**
 - System doesn't need to "re-discover" the road layout
 - It only needs to identify dynamic elements (pedestrians, other cars)
- **Static vs. Dynamic Split**
 - **Static Environment:** Provided by the HD map
 - **Dynamic Perception:** Handled by real-time sensors (LiDAR, Radar, Cameras)
- **Increase reliability:**
 - Better HD maps reduce the depend on real-time sensor capture during complex scenarios



To organize complex data, HD maps are structured into five distinct layers:

Base Map	SD map that includes road curvatures, elevation, and GPS coordinates
Geometric	3D point cloud created from the mapping sensors (LIDAR, camera, GPS, etc.) that includes the stationary objects in the surroundings
Semantic	2D or 3D semantic objects (such as lane markings, street signs, and other details)
Map Priors	Prior information and entity behavioral data, such as average cycle/pedestrian speeds, and SPaT information
Real-time	Real-time traffic information that a CAV can receive (V2X) to assist with trip planning and navigation

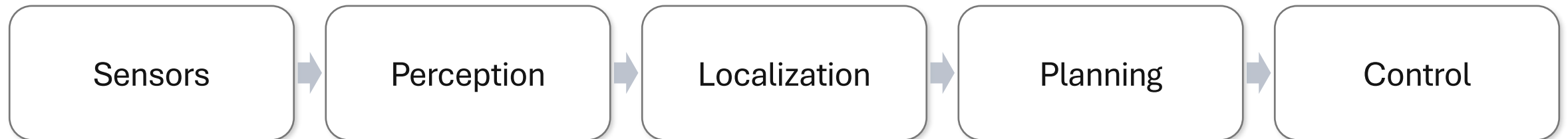
- Where to Store the Data?
 - On-board vehicle, Cloud, Network Operations Center (NOC), Edge



- Sensing
 - Obtain precise information regarding the **position** and **heading** of the CAV
 - Relative to a 3D HD map
 - Different from “perception for object detection” (covered later)
- Why sensing is hard (operating conditions)
 - Sensing must remain functional and accurate across
 - Low lighting, Tunnels, Urban canyons (tall buildings), Varying weather conditions (rain, fog, etc.)
- Multi-sensor data → Sensor Fusion requirement
 - Localization uses **multiple sensors**, each producing **different formats** of data
 - Examples: point clouds, images, ranges, velocities, inertial signals, wheel ticks
 - **Sensor Fusion**: Combining data from multiple formats into a single, cohesive world model
 - Sensors can degrade differently → need redundancy + robust fusion



- Sensor technologies used in localization are either:
 - **Exteroceptive**: sense the external surroundings (outside the vehicle)
 - **Proprioceptive**: sense the vehicle's internal motion/state (inside the vehicle)
- Key distinction
 - Exteroceptive sensors are used in mapping + sensing stages
 - Proprioceptive sensors are used only in sensing (for motion/odometry support)
- **Sensor: Passive or Active**
 - **Passive**: camera
 - **Active**: radar/LiDAR/ultrasonic



- **Sensor hardware: the 6 questions**
 - To predict **performance, failure modes, and cost/compute needs**
- **What physical quantity is measured?**
 - **Camera:** photons / intensity (RGB/IR)
 - **Radar:** RF reflections (phase/frequency)
 - **LiDAR:** time-of-flight of light
 - **IMU:** acceleration + angular rate
 - **Note:** the physics determines what the sensor can “see” and what it cannot
- **Passive or active?**
 - **Passive:** camera (depends on lighting).
 - **Active:** radar/LiDAR/ultrasonic (emits energy).
Note: active helps in darkness, but brings interference, eye-safety/regulation, and power considerations

Terminology

- FOV = field of view
- TOF = time-of-flight
- SNR = signal-to-noise ratio
- RCS = radar cross section



- **Sensor hardware: the 6 questions**
 - To predict **performance, failure modes, and cost/compute needs**
- **What is the measurement model?**
 - Example outputs:
 - **Camera:** bearing / pixels / intensity (2D)
 - **Radar:** range + Doppler + angle (often sparse, noisy)
 - **LiDAR:** 3D point cloud: range + angle (geometry-rich)
 - **Note: Perception must match the data type** (pixels vs points vs range–Doppler)
- **What limits SNR (and therefore accuracy)?**
 - **Camera:** low light, glare, fog/rain, motion blur
 - **Radar:** multipath, clutter, low RCS objects, interference
 - **LiDAR:** rain/fog, sun/retroreflectors, surface reflectivity
 - **Note:** SNR drives range/precision and failure probability.

Terminology

- FOV = field of view
- TOF = time-of-flight
- SNR = signal-to-noise ratio
- RCS = radar cross section



- **Sensor hardware: the 6 questions**
 - To predict **performance, failure modes, and cost/compute needs**
- **Sampling limits (frame/chirp rate, latency, sync)?**
 - Even perfect sensors fail if too slow or unsynchronized
 - Frame rate / chirp rate affects tracking of fast objects
 - Latency affects control and safety distances
 - Time synchronization matters for sensor fusion (camera–radar–LiDAR alignment)
 - Quick example: At 20 m/s, a 100 ms delay means the car moves 2 meters before you act
- **How does it fail, and how do we detect failure?**
 - **Failure modes:** occlusion, saturation, dropped frames, miscalibration, spoofing/interference, hardware degradation.
 - **Detection ideas:** sensor health checks, consistency checks across sensors, confidence metrics, redundancy, plausibility filters.

Terminology

- FOV = field of view
- TOF = time-of-flight
- SNR = signal-to-noise ratio
- RCS = radar cross section



- **Sensor hardware: the 6 questions**
 - To predict **performance, failure modes, and cost/compute needs**
- **Key metrics**
 - **Range:** How far the sensor can reliably detect an object
 - **Resolution:** How small a difference it can distinguish (distance/angle/speed)
 - **Accuracy:** How close the measurement is to the true value (systematic error)
 - **Precision:** How consistent repeated measurements are (random noise)
 - **Latency:** How long it takes from sensing to producing an output you can use
 - **Coverage:** What area the sensor can see (FOV + mounting + occlusions)
 - **Robustness:** How well it works in difficult conditions (weather, dirt, vibration, interference)
- Range $\uparrow$ usually needs power / better optics / better antenna / larger aperture
- Resolution $\uparrow$ increases compute and bandwidth
- Robustness $\uparrow$ often needs redundancy + fusion + better hardware



- Passive Sensor
 - Cameras are passive optical sensors
 - Cameras do not emit energy (unlike Radar/LiDAR) but capture ambient light
- Primary Strengths
 - Cameras provide high-fidelity edges, textures, and color data
 - Essential for semantic awareness (e.g., distinguishing a cyclist from a pedestrian)
- Role in AVs
 - Serve as the primary tool for object recognition and lane/sign classification.



- Dimensionality Mapping:
 - Camera projects a 3D point (X, Y, Z) in the real world onto a 2D image plane (u, v)
- 3D Coordinate System (X, Y, Z)
 - X and Y : Horizontal and vertical position relative to the camera lens center
 - Z : Critical **distance (depth)** from the camera sensor to the object
- 2D Coordinate System (u, v)
 - Definition: (u, v) are pixel coordinates used to locate any point within a digital image
 - (u) Horizontal: Column index or pixel position along the width of the image
 - (v) Vertical: Row index or pixel position along the height of the image



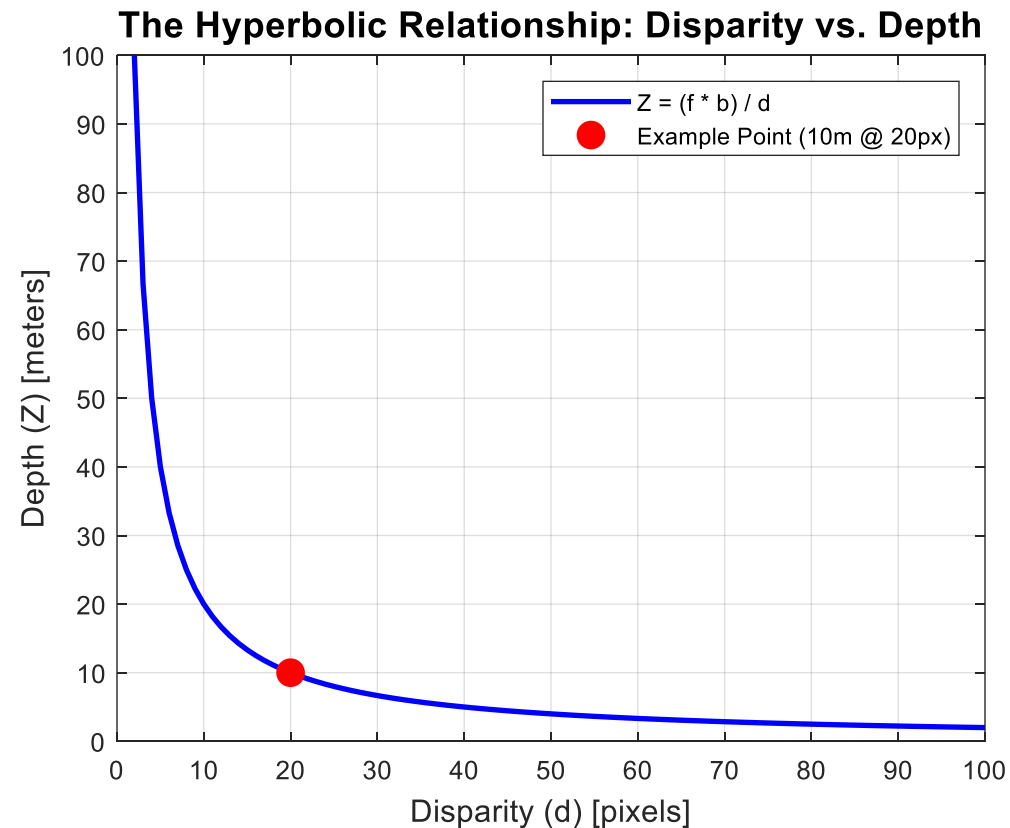
- **Primary Strengths: Semantic Awareness**
 - Rich **2D image**: Provides high-fidelity edges, textures, and color data
 - Good object recognition : classification a cyclist from a pedestrian
- Core Problem: The Missing Dimension (Z)
 - **Dimension Loss**: Camera projects a 3D point (X, Y, Z) onto a 2D image plane (u, v)
 - **Depth Ambiguity**: Objects appear smaller as distance (Z) increases
 - Due to perspective, absolute depth cannot be recovered from a single 2D view
- **Inferencing Depth**: Sensor provides no direct depth data, systems must predict
 - **Geometric Cues**: object size, lane width, vanishing point
 - **Motion cues**: optical flow, structure-from-motion
 - **Priors**: known/typical object dimensions (e.g., "a car is X meters wide")



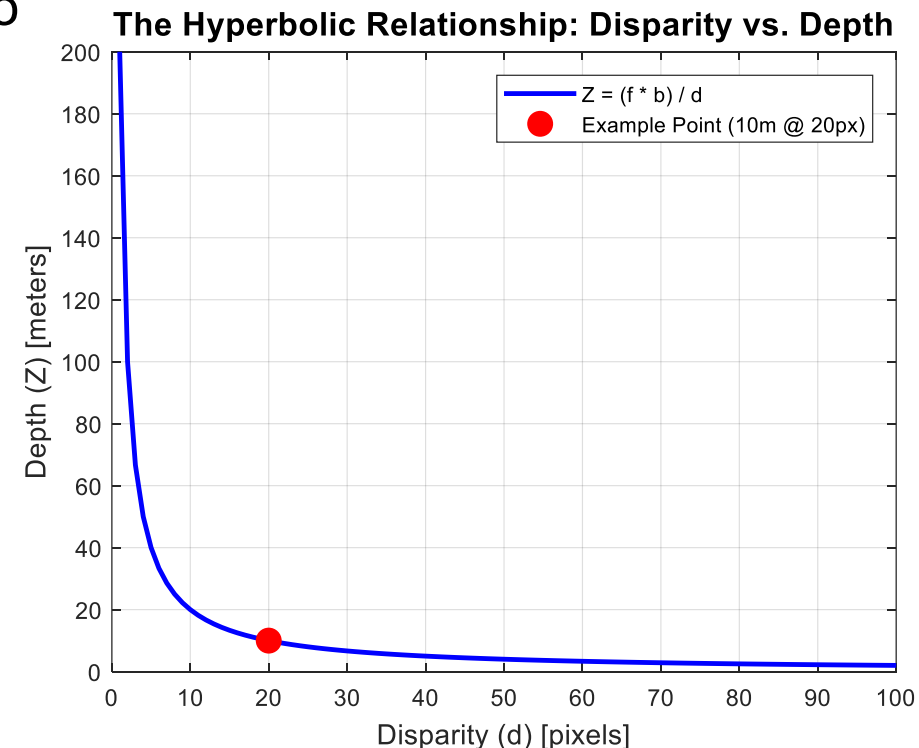
- **Primary Strengths: Direct Depth Perception**
 - Native 3D Sensing: Provides dense depth maps without relying on prediction
 - Can calculate distance to unknown or "rare" objects
 - High Redundancy: Two mono-cameras simultaneously providing 3D spatial data.
- **Solution: Recovering the Dimension (Z)**
 - Principle of Disparity: Identifying the horizontal shift of pixels d between cameras
 - The left u_L and right u_R images (u, v): $d = u_L - u_R$
 - Triangulation: Calculates the absolute depth (Z) using the known geometry of camera
 - Formula for depth: $Z = \frac{f \cdot b}{d}$ f = focal length, b = baseline, d = disparity
 - f (Focal Length): The distance between the lens and the sensor
 - b (Baseline): The fixed physical distance between the two camera centers
 - d (Disparity): The difference in image location of the same object



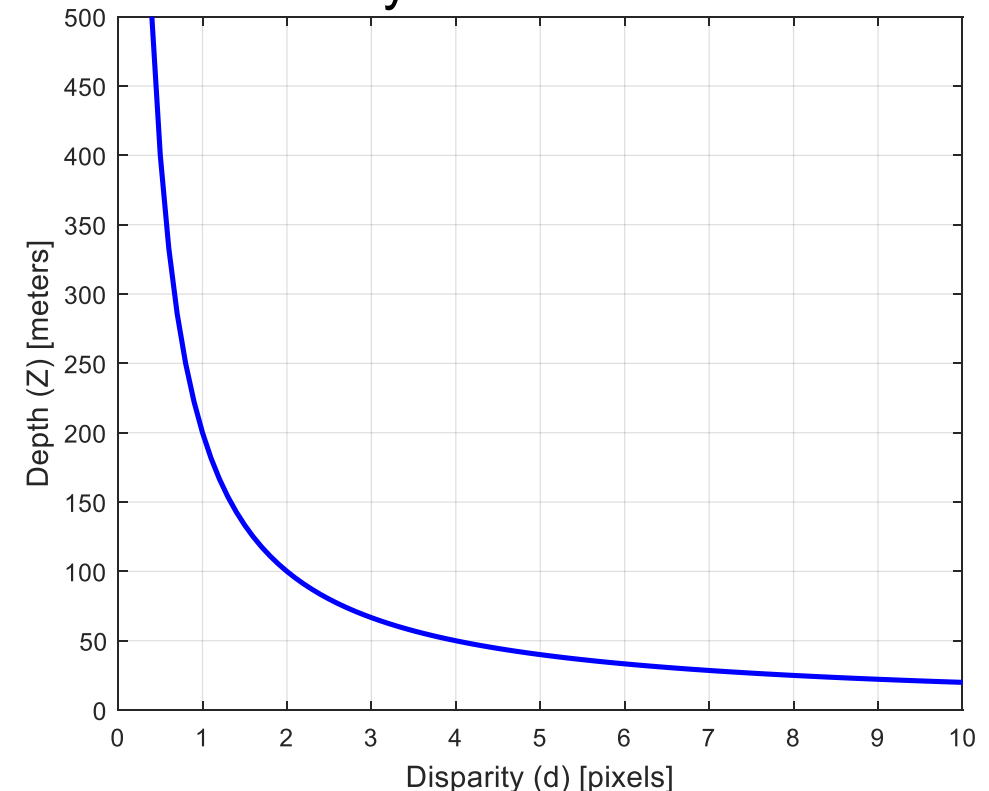
- Formula: $Z = \frac{f \cdot b}{d}$ f = focal length, b = baseline, d = disparity
 - Scenario: A stereo system with a 1000px focal length f and a 0.2m baseline b
 - Observation: The system detects a pixel disparity d of 20px, $Z=10$ meters



- Inverse Relationship (Hyperbola)
 - High Disparity (d) = Low Depth (Z):
 - When object is very close to the car, the difference in its position between the left and right camera images is huge. On the graph, this is the area where the curve is very low and flat.
 - Low Disparity (d) = High Depth (Z):
 - As an object moves further away, its position in the two images becomes almost identical, making the disparity (d) approach zero



- Sensitivity and Precision
 - "Steep" Zone (Low Disparity):
 - Curve shoots upward as d gets small (e.g., between 1 and 10 pixels).
 - At these distances, a tiny change in disparity results in a massive change in depth
 - Example: If the sensor miscalculates the disparity by just 1 pixel when the object is far
 - The car might think an obstacle is at 100 meters when it is actually at 150 meters.
 - The "Flat" Zone (High Disparity):
 - Disparity is high (close objects)
 - System is very precise
 - 1-pixel error changes the depth a few cm only



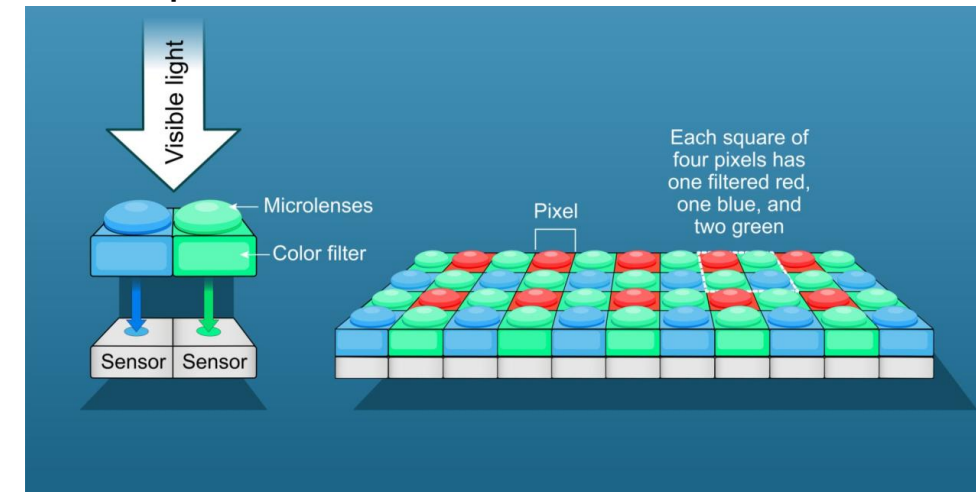
- **Calibration Criticality:** Two cameras must remain perfectly aligned
 - Even a tiny physical shift or vibration can ruin the depth calculation
- **Computationally Heavy:** Matching millions of pixels between the left and right images
 - Matching in real-time requires significant image processing power (ISP/GPU)
- **Baseline Selection:**
 - Short Baseline: Ideal for wide-angle, close-range tasks like automated parking
 - Long Baseline: Essential for high-speed highway sensing to "see" further ahead



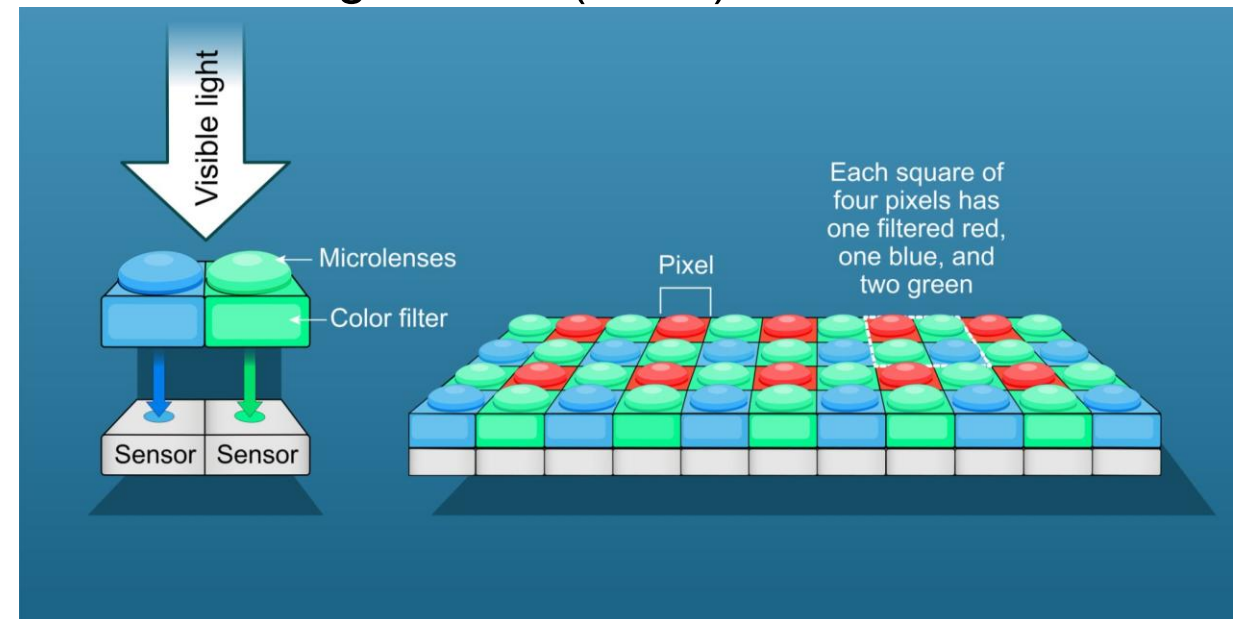
- Light Quantity: Aperture
 - Controls the amount of light hitting the sensor
 - Cameras often use fixed apertures optimized to balance between low-light and Depth of Field
 - If the lens aperture is too small, not enough photons reach the sensor, especially at night
- Light Quality: Lens Aberrations
 - Cheap lenses introduce distortions like chromatic aberration (color fringing at edges) or spherical aberration (blurriness).
 - If a pedestrian looks blurry to the camera, the AI might misclassify them as a tree.
- Narrow FOV = Long Range
 - Lens pixels are concentrates, high detail on distant objects.
- Wide FOV = Short Range
 - Lens pixels are stretched to cover a massive panoramic area.
 - Pixels are spread so thin over a wide angle, you lose the ability to see fine detail far away.



- **Light Capture (The Microlenses)**
 - Before the light hits the sensitive part of the sensor, it passes through **microlenses**.
 - **Microlenses** are tiny, individual lenses built on top of each pixel
 - Used to focus as many incoming photons as possible directly into the photodiode
 - Prevent light from leaking into adjacent pixels
- **Light Filtering (The Bayer Filter)**
 - Photodiodes only measure the *intensity* (quantity) of light and not its colour
 - Sensor is inherently color-blind.
 - **Color Filter Array (CFA)**, arranged in a **Bayer pattern**, is placed above the photodiodes.
 - Each 2 X 2 square contains one Red, one Blue, and two Green pixels



- Why Green is Doubled in the Bayer Pattern (RGGB):
 - Human eye is more sensitive to green light and perceives higher detail in green wavelengths
 - Helps AI algorithms better distinguish edges and details.
 - Double-green pattern provides luminance map which helps AI to calculate the edge position
- Digital Conversion: **Photon-to-Electron**
 - When photons pass through the filter and hit the silicon photodiode, they release electrons
 - The number of electrons released is directly proportional to the light intensity.
 - Electrical charge is measured and converted into the digital data (RAW) that the AI will eventually process.



- Rolling Shutter: Line-by-Line Scanning
 - Scan and read the image sensor pixel row by pixel, or line-by-line.
 - Sensor sends a "reset" signal to **Row 1**, then a fraction of a millisecond later to **Row 2**, and so on
 - Top and bottom image is captured at a different time
 - Fast-moving objects appear tilted or leaning, cause **Geometric Distortion**.
 - Confuse computer vision tracking algorithms, object's shape doesn't match the AI's trained models.
- Global Shutter: Instantaneous Capture
 - Captures the entire frame at **exactly the same instant**.
 - Best for **Sensor Fusion** eliminates temporal distortion, timestamp matches LiDAR and Radar
- Trade-off: Global shutter are more expensive
 - Often have lower overall resolution compared to rolling shutter sensors.
 - sensor must store the exposure value for every pixel simultaneously
 - each pixel requires more transistors and storage components
 - extra sensor chip space makes less room for light-sensitive photodiodes.
 - To fit this complex circuitry, the pixels must be larger, or fewer pixels can be crammed onto a sensor of the same size, resulting in a lower maximum resolution.



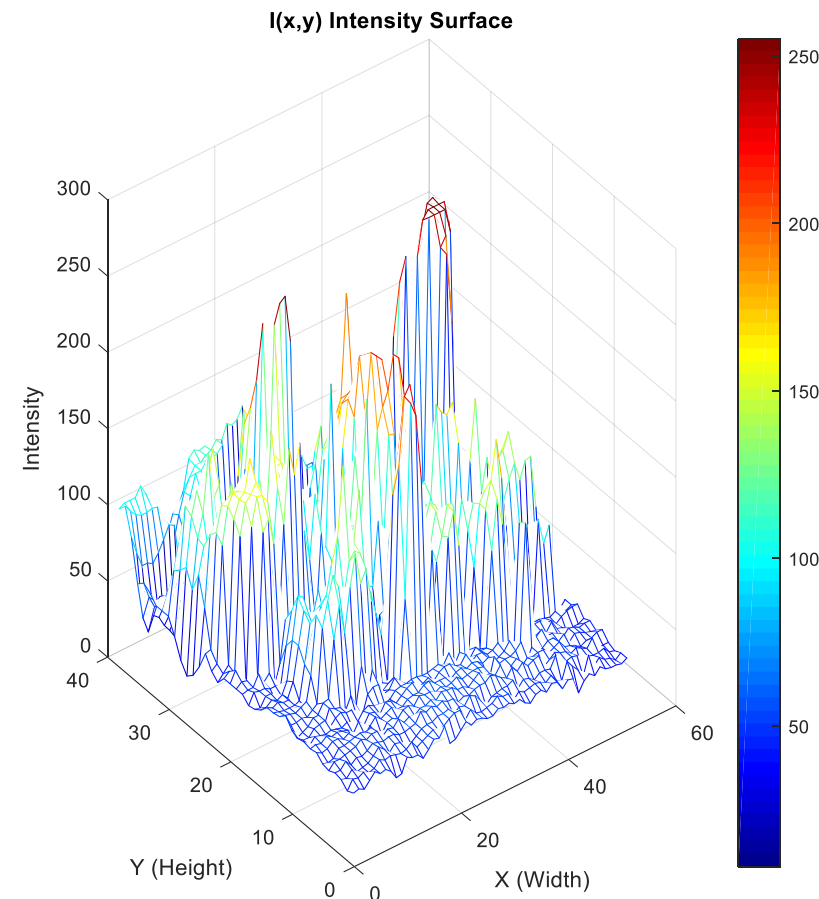
- **Demosaicing (Color Reconstruction)**
 - Sensor uses a Bayer filter, each individual pixel only measures **one** color intensity
 - **ISP** looks at the known color values of the surrounding pixels.
 - For a red pixel, ISP estimates green and blue values from nearby green and blue pixels
- **Denoising:** ISP utilizes **spatial filtering algorithms**
 - Remove electrical noise (graininess) often found in low-light conditions
 - ISP analyze a pixel against its neighbors
 - If a pixel is significantly brighter or darker than its neighbors without a clear edge, it is likely noise
- **Sharpening:** ISP uses **high-pass filtering** algorithms
 - Artificially increases contrast specifically at object boundaries (edges)
- **Tone Mapping:**
 - Compresses **High Dynamic Range (HDR)** data into a lower, usable range of brightness
 - Adjusts bright areas and dark areas independent so details in both are visible simultaneously.



- **Color Space Transition (YUV Conversion):**
 - While ISP works in RGB, many AI models convert this to **YUV** (Luma/Chroma)
 - YUV separates brightness (Y) from color (U/V)
 - Allowing the system to prioritize structural details (luminance) while saving bandwidth on color.
 - By focusing on the "Y" (Luma), system can detect objects faster and with higher precision.
 - Color information (U/V) is less critical for detecting objects, and it can be **compressed**
- **Deep Learning Inference:**
 - High-quality RGB/YUV images are fed into neural networks (like CNNs)
 - Extract features and classify objects such as pedestrians, cyclists, and traffic signs.
- **The Fusion Layer:**
 - Multi-Modal Integration: Camera data is fused with LiDAR (depth) and Radar (velocity) to compensate for individual sensor weaknesses.
 - Temporal Tracking: The system uses sequential frames to predict the future trajectory of moving objects, essential for safe maneuvering.

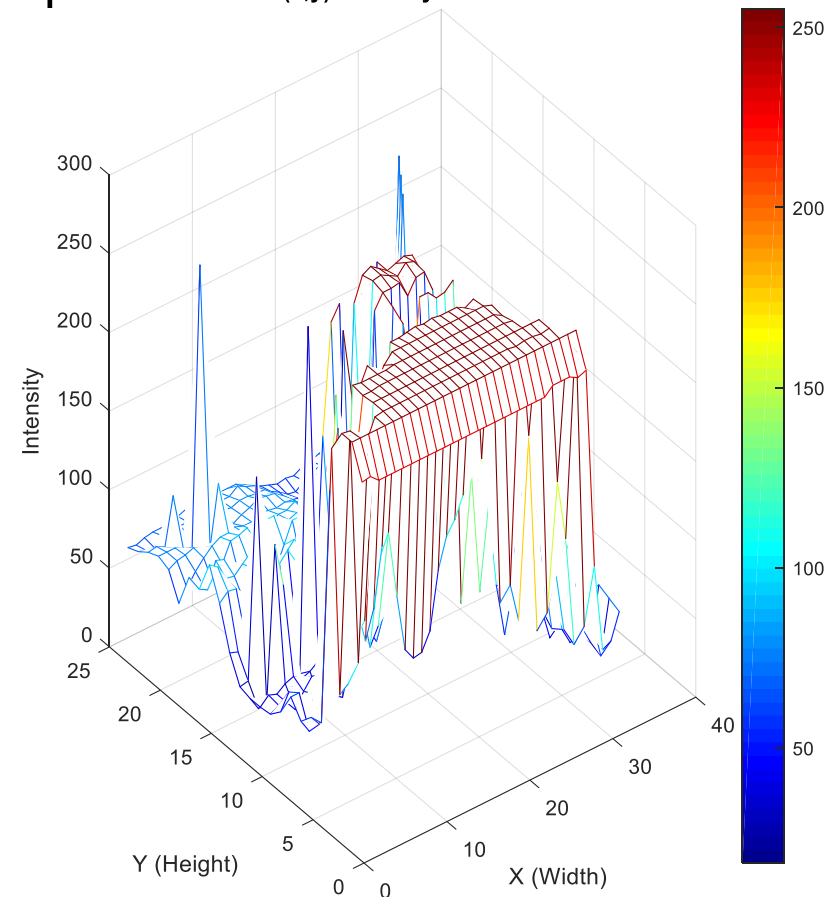


- Image Processing
 - AI processes a scene, it treats the image as a 3D surface where height represents intensity
 - An image $I(x,y)$ maps coordinates to brightness values
 - Neural networks don't "see" objects; they find patterns in these numerical "landscapes"



- Image Processing
 - AI processes a scene, it treats the image as a 3D surface where height represents intensity
 - An image $I(x,y)$ maps coordinates to brightness values
 - Neural networks don't "see" objects; they find patterns in these numerical "landscapes"

2D Grayscale Image

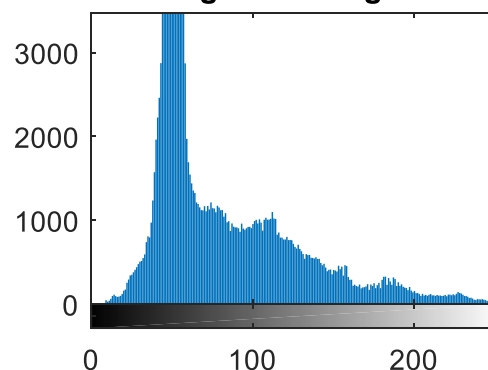


- Lighting varies wildly in driving. We use histogram equalization to standardize the input for the AI.
- algorithm remaps pixel values based on their cumulative distribution
- Reduces the influence of lighting conditions and improves the robustness of later detection steps

Original Low Contrast



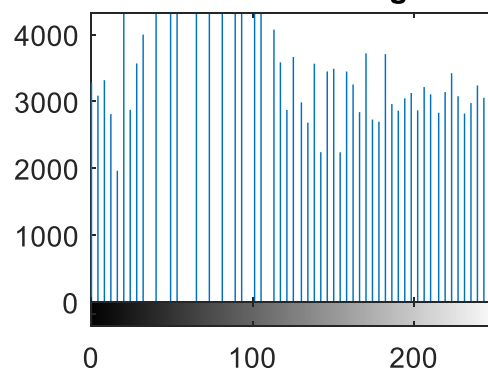
Original Histogram



Equalized (Normalized)



Normalized Histogram

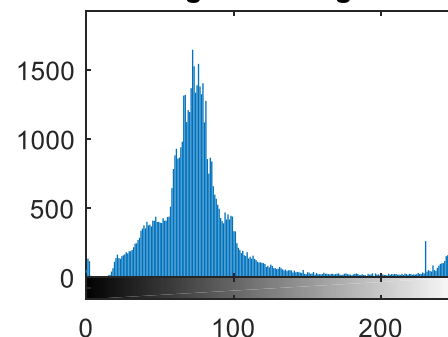


- Lighting varies wildly in driving. We use histogram equalization to standardize the input for the AI.
- algorithm remaps pixel values based on their cumulative distribution
- Reduces the influence of lighting conditions and improves the robustness of later detection steps

Original Low Contrast



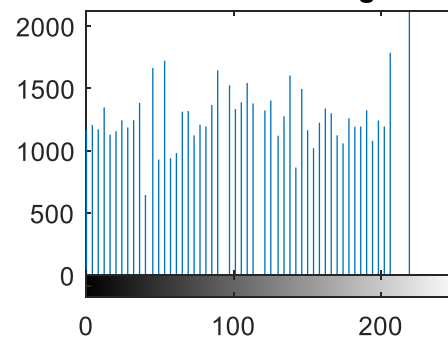
Original Histogram



Equalized (Normalized)



Normalized Histogram



- To park or stay in lanes, the AI must transform the 2D image coordinates into 3D world planes
- Homography: A 3 X3 matrix H transforms a point x to x' ($x' = Hx$)
- Remapping four camera views into a single "Bird's-Eye View" for parking assistance.

Original Perspective



Warped (Projective Transformation)



- To park or stay in lanes, the AI must transform the 2D image coordinates into 3D world planes
- Homography: A 3 X3 matrix H transforms a point x to x' ($x' = Hx$)
- Remapping four camera views into a single "Bird's-Eye View" for parking assistance.

Original Perspective



Warped (Projective Transformation)



- Convolution is the core of feature extraction, using a sliding window and a kernel $K(i,j)$
- Gaussian Blur: Uses a smoothing kernel to remove high-frequency noise
- Laplacian Edge Detection: Highlights sharp transitions in intensity to find object boundaries

Input $I(x,y)$



Gaussian Blur (Noise Reduction)



Laplacian (Edge Extraction)



- Convolution is the core of feature extraction, using a sliding window and a kernel $K(i,j)$
- Gaussian Blur: Uses a smoothing kernel to remove high-frequency noise
- Laplacian Edge Detection: Highlights sharp transitions in intensity to find object boundaries

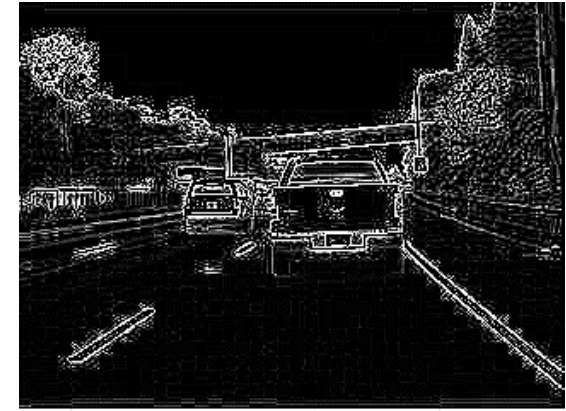
Input $I(x,y)$



Gaussian Blur (Noise Reduction)

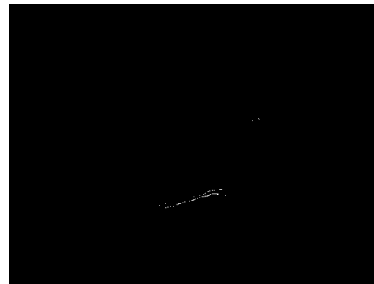


Laplacian (Edge Extraction)



- The ultimate goal is identifying individual objects through segmentation masks
- **Color Thresholding:** Fast masking based on color ranges (e.g., finding the blue car body)
- **Bounding Boxes:** Fitting a minimum area rectangle to a contour to determine orientation and ground contact points

Binary Segmentation Mask

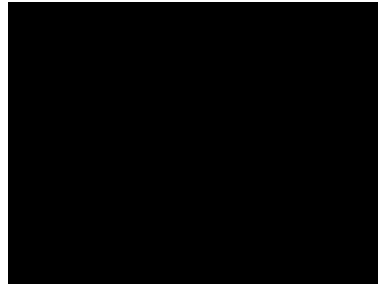


Object Contours for AI Perception



- The ultimate goal is identifying individual objects through segmentation masks
- **Color Thresholding:** Fast masking based on color ranges (e.g., finding the blue car body)
- **Bounding Boxes:** Fitting a minimum area rectangle to a contour to determine orientation and ground contact points

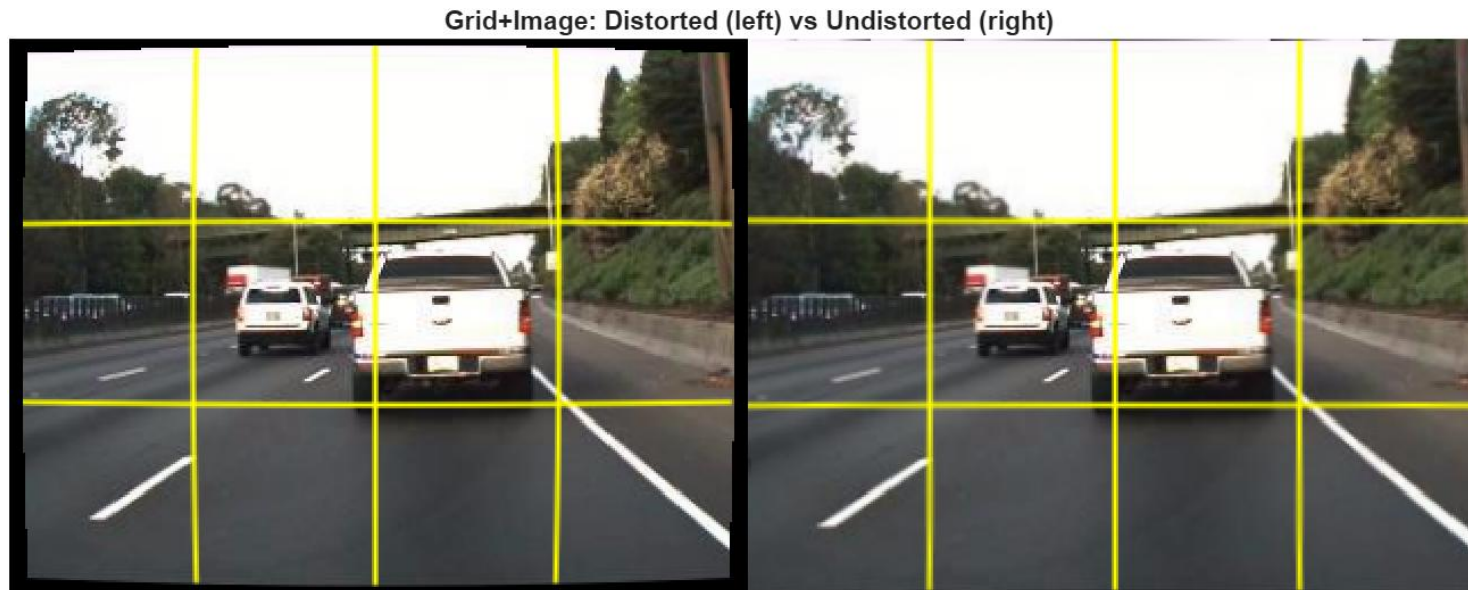
Binary Segmentation Mask



Object Contours for AI Perception



- Cameras lenses may introduce radial and tangential distortion
 - Cause straight lines (like lane markings) to appear curved
 - Use **Camera Intrinsics** and **Distortion Coefficients** to rectify the frame
 - Ensures spatial measurements (object distance and angle) are reliable



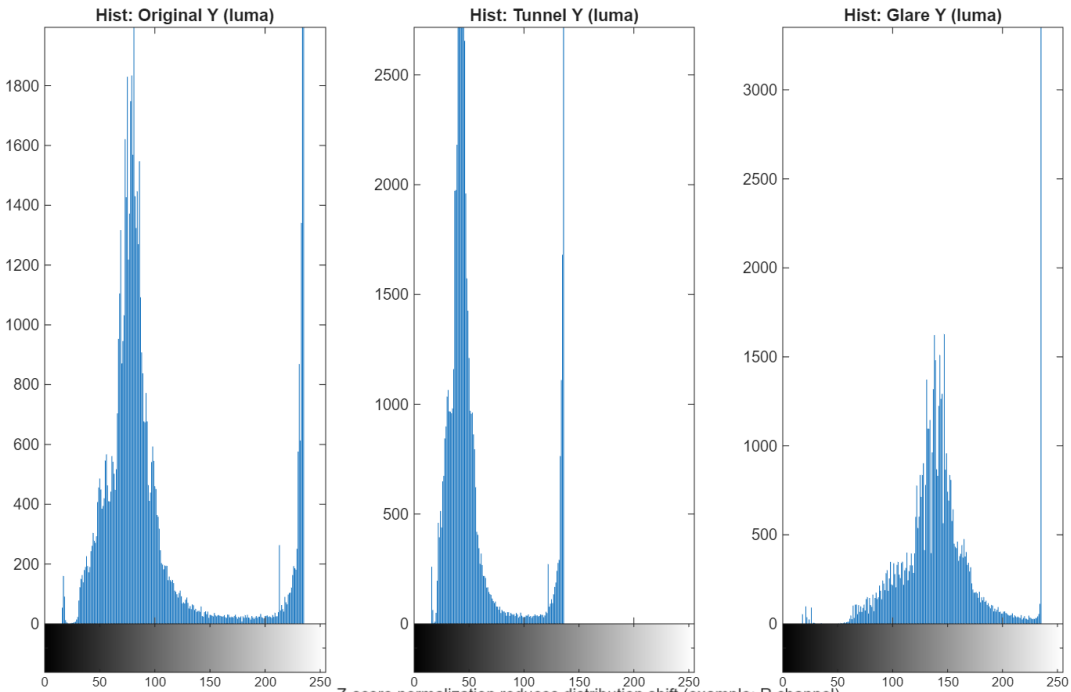
- Drastic changes in lighting create unstable pixel intensity distributions
 - Leading to inconsistent AI performance
 - (e.g., entering tunnels, glare)
- Pre-processing
 - Ensure pixel intensities are consistent, regardless of original image was bright or very dark
 - **Z-score Normalize:** $x' = (x - \mu) / \sigma$
 - Original Pixel Value (x): raw intensity value of a specific pixel (0 to 255 in a standard grayscale)
 - Mean (μ): average intensity of all pixels in the image (or a specific batch).
 - It represents the "center" of the brightness
 - Standard Deviation (σ): measure the "spread" or contrast of the pixel values
 - low σ means the image has low contrast; a high σ means a wide range of tones
 - Normalized Pixel Value (x')
 - Centering ($x - \mu$): shift the distribution so that the new average is **zero**
 - Scaling ($/\sigma$): "squash" or "stretch" the distribution so that the new standard deviation is **1**



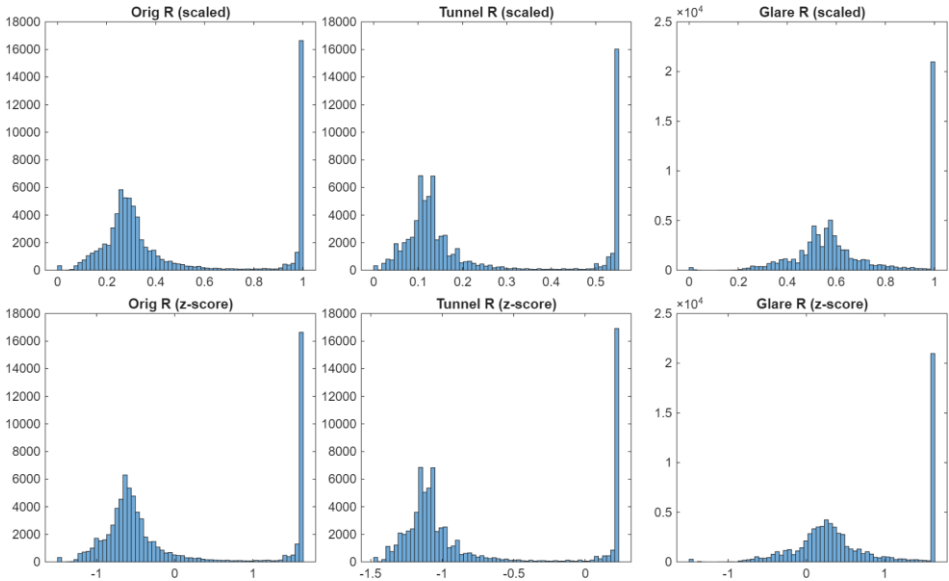
Lighting shift: Original (left) vs Tunnel-like (right)



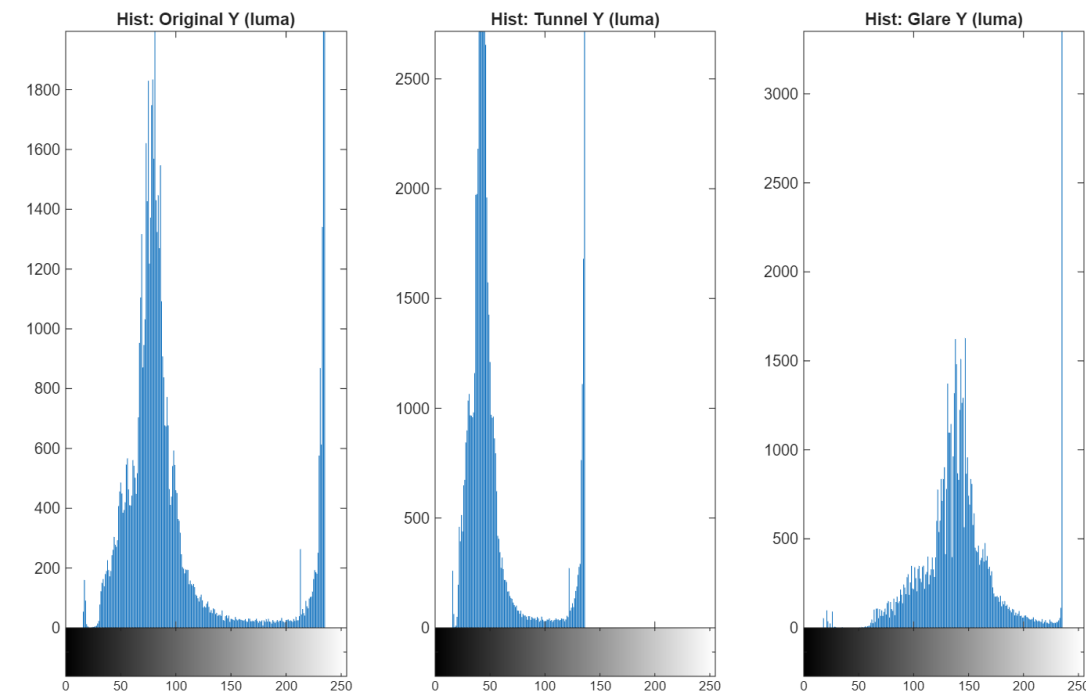
Lighting shift: Original (left) vs Glare-like (right)



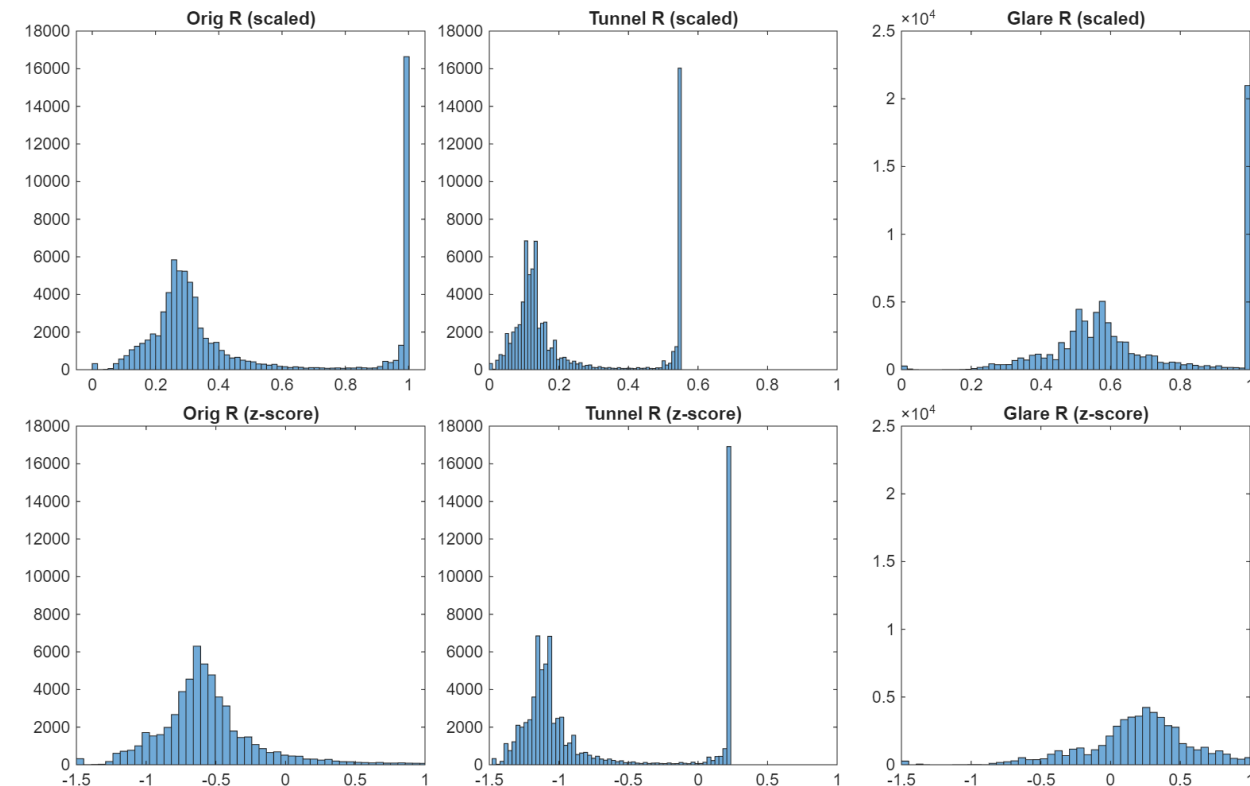
Z-score normalization reduces distribution shift (example: R channel)



- Lighting shift
 - Tunnel makes everything darker and glare makes highlights saturate
- Histograms
 - X-axis is intensity 0–255, Y-axis is number of pixels
 - **Tunnel** histogram shifts toward **lower intensities**
 - **Glare** histogram shifts toward **higher intensities**
 - **Distribution shift** impacts inference reliability

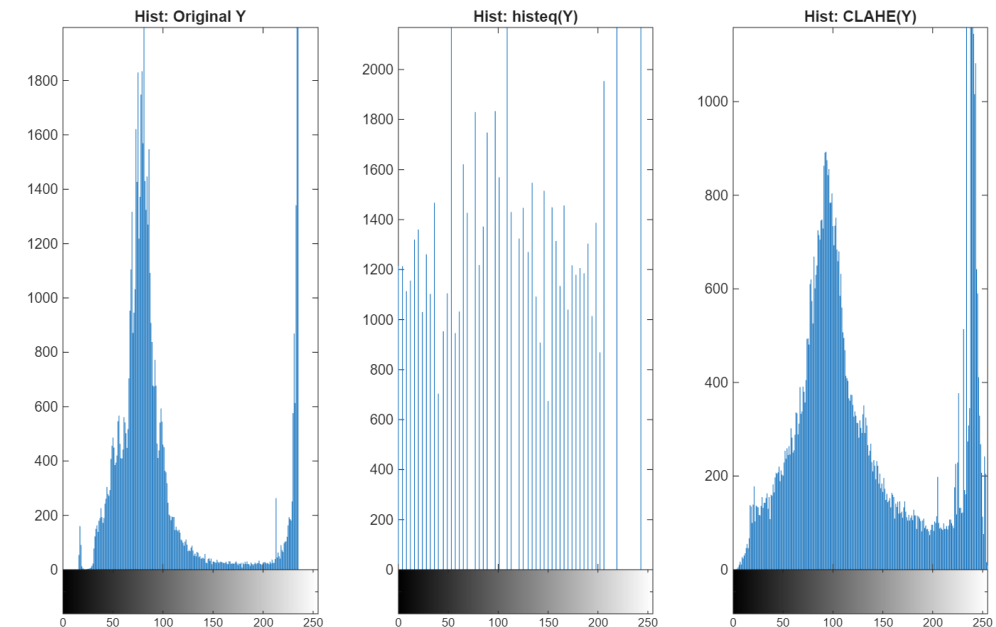


- Before normalization
 - Distributions (scaled $[0,1]$) are very different for original/tunnel/glare
- **z-score normalization**
 - Distributions are centered around 0 and scaled by standard deviation
- Normalization reduces sensitivity to brightness
 - Makes the input scale consistent

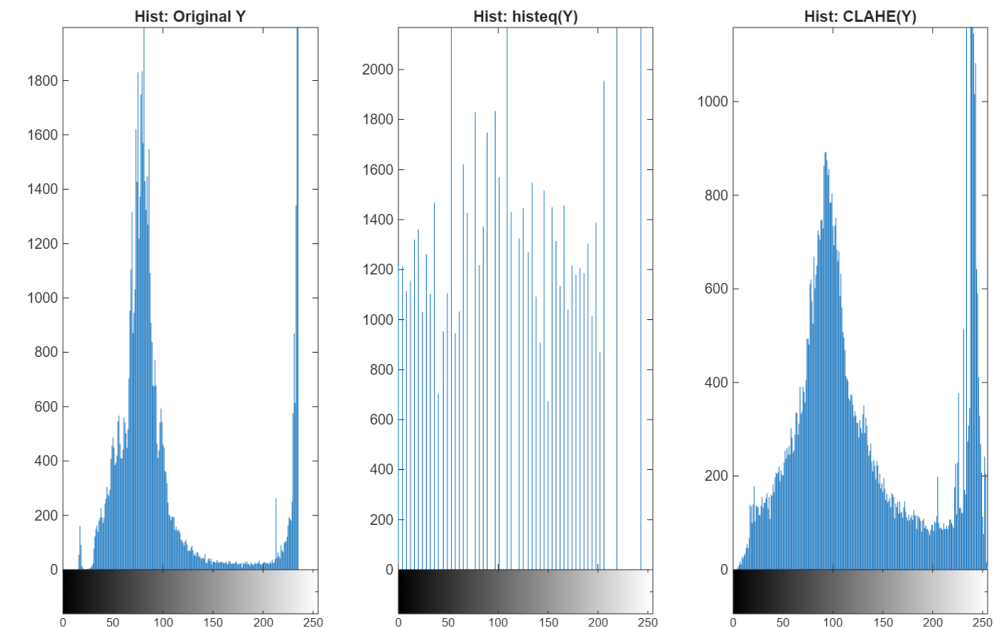


- Histogram Equalization vs CLAHE
 - Beyond scaling, these techniques aim to improve visibility by redistributing pixel intensities
- **Global Histogram Equalization:**
 - Applies a single mapping function to the entire image
 - Downside: Can look harsh, lose detail in highlights, and amplify background noise

Contrast: Histogram Equalization (left) vs CLAHE (right)



- Histogram Equalization vs CLAHE
 - Beyond scaling, these techniques aim to improve visibility by redistributing pixel intensities
- **CLAHE (Contrast Limited Adaptive Hist. Eq.):**
 - Local Processing: Operates on small blocks to handle local lighting variations
 - Clip Limit: Prevents over-amplification of noise in uniform areas
 - Result: Improves visibility in dark regions while preserving a natural, realistic appearance



- Resizing
 - Images are resized to fit the model's fixed input shape
 - Example: Scaling a 1920 X 1080 camera frame to a 640 X 384 model input
- Letterboxing (Aspect Ratio Preservation):
 - Adds padding to prevent squashing effect
 - Resizing based on the **minimum scale factor** and padding the rest with zeros (black pixels)
 - Avoid confusing the classifier

Original Frame
(320 x 240)



Stretched to 640 x 384
(Geometric Distortion!)



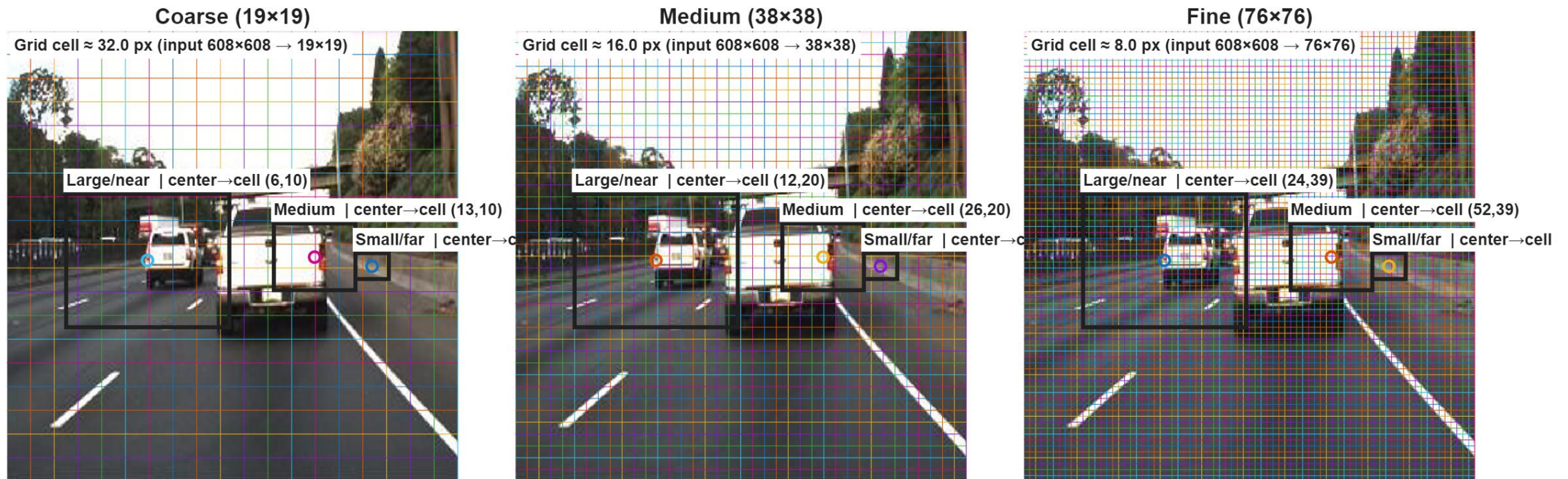
Letterboxed to 640 x 384
(Aspect Ratio Preserved)



- YOLO (You Only Look Once)
 - A popular, real-time object detection algorithm
 - Identifies and classifies multiple objects in images or videos in a single pass
 - YOLOv4 operates as a single convolutional neural network
 - Performing inference in one forward pass through the network
 - Latest and most advanced version of the YOLO series is **YOLO26**
- YOLOv4 analyze image by **3 feature map scales**
 - **Coarse scale** (small grid like 19×19 or 13×13)
 - zoomed-out view → good for big/near objects
 - **Medium scale** (38×38 or 26×26)
 - mid zoom → good for medium objects
 - **Fine scale** (76×76 or 52×52)
 - zoomed-in view → good for small/far objects



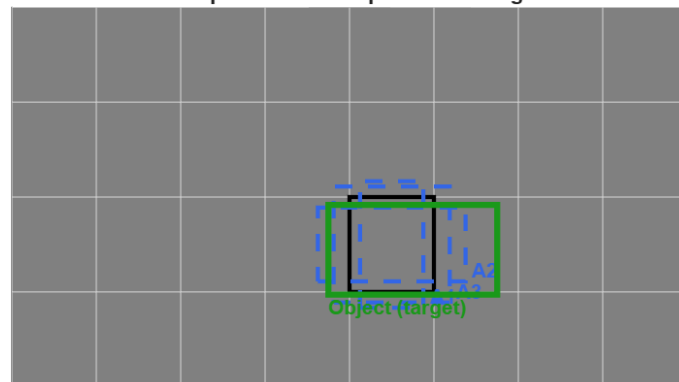
- Feature Map Extraction (Backbone)
 - Feature map is divided into an SXS grid of cells
 - Each grid cell is mapped to a specific area of the original image
- **Center Point Mapping:**
 - If the center point of an object's ground-truth bounding box falls within a specific grid cell
 - that cell is directly **responsible** for predicting that object



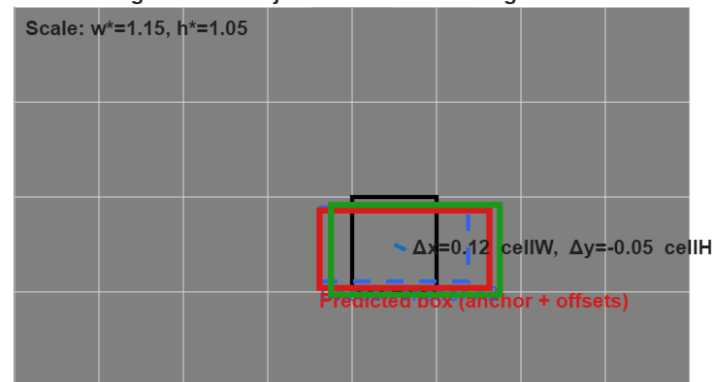
- Anchor Boxes
 - uses "priors" (pre-defined box shapes)
 - Instead of guessing a rectangle, it calculates the **offsets** from these anchors
 - For each grid cell, the network predicts K=3 anchor-based candidates
- YOLO places blue dashed rectangles: A1, A2, A3
 - These anchors are **different shapes** (tall, wide, square)
 - The **green box** is the **true object** (target / ground-truth box)
- YOLO decides **which anchor is the best match** for the object
 - One with highest objectness / best overlap (blue dashed)

Anchor Boxes in YOLO: Templates + Learned Offsets

Left: Multiple anchor templates at one grid cell



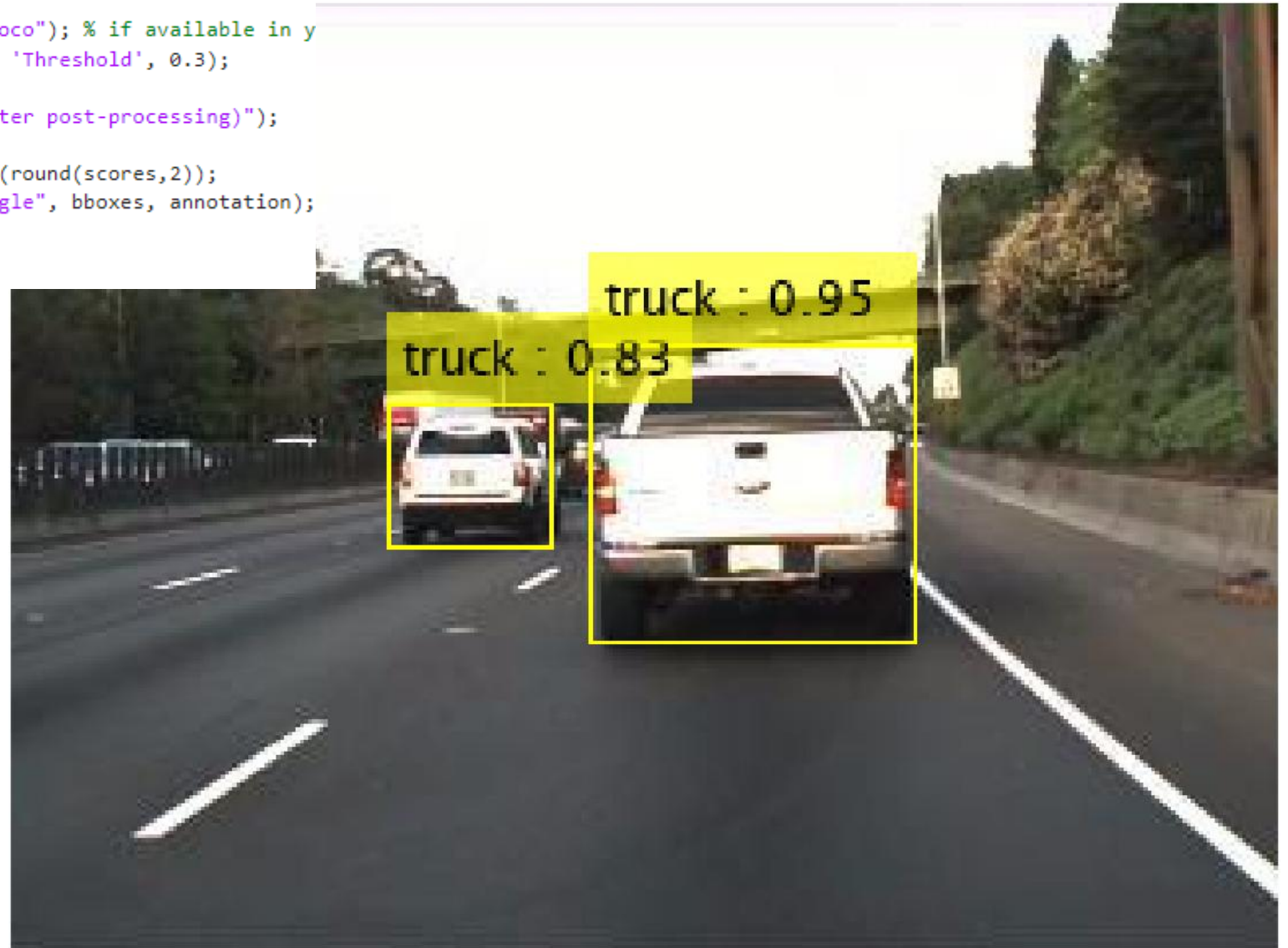
Right: YOLO adjusts one anchor using offsets

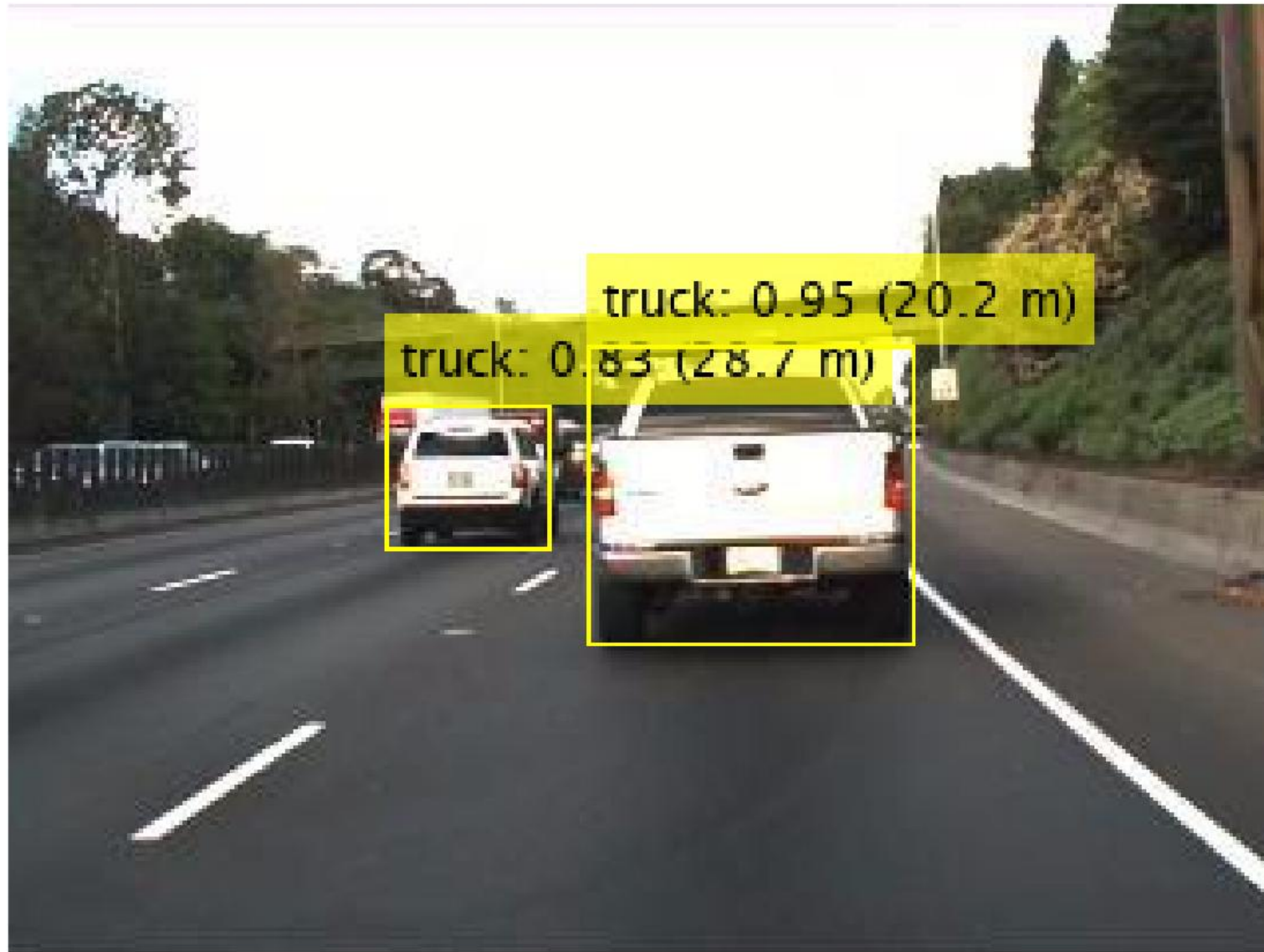


- **CSPDarknet53** backbone
 - uses 53 convolutional layers to extract a high-dimensional feature map
 - Representing shapes, not just pixels
 - **Layer 1-10:** Learns edges and colors.
 - **Layer 11-30:** Learns shapes (circles, rectangles).
 - **Layer 31-53:** Learns complex objects (wheels, pedestrians).
- Prediction at each grid cell
 - Bounding Box Coordinates: Predicts x, y, w, h (offsets relative to the cell)
 - Object Score: probability that a physical object exists in that cell.
 - Class Probabilities: If an object exists, is it a Car? Truck? Pedestrian?
- Non-Maximum Suppression
 - Multiple grid cells detect the same object
 - compares the overlapping boxes and keeps only the box with the highest confidence score

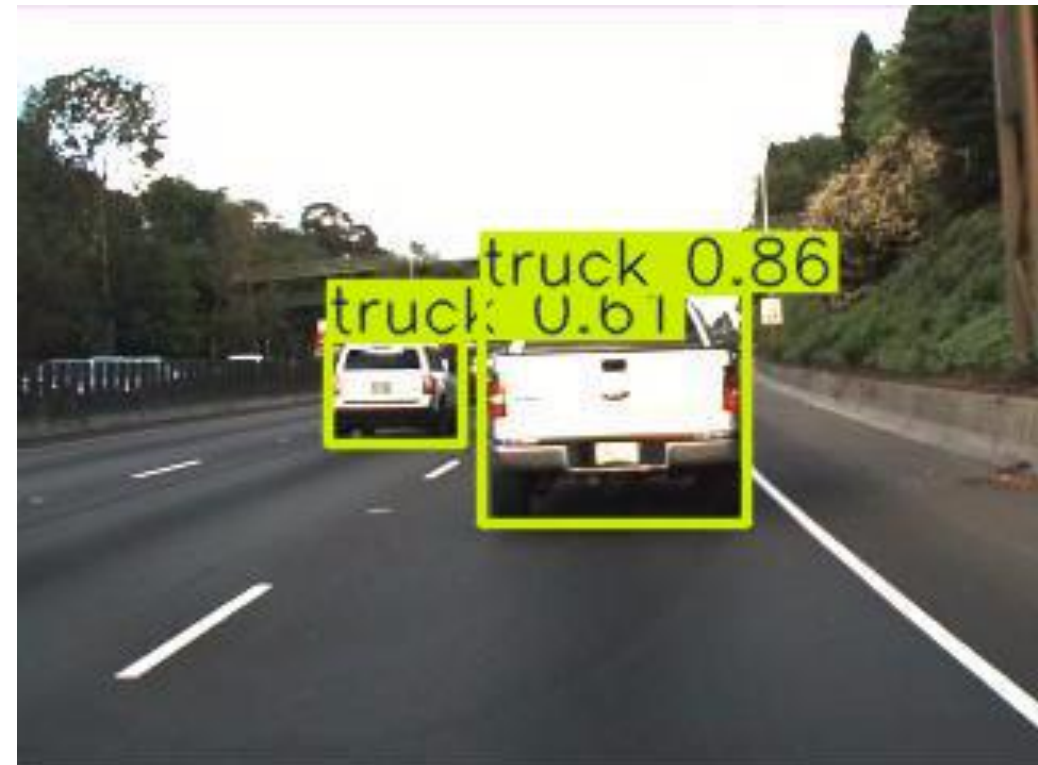


```
1  img = imread("highway.png");
2
3  % img = imread("highway.png");
4  detector = yolov4ObjectDetector("csp-darknet53-coco"); % if available in y
5  [bboxes, scores, labels] = detect(detector, img, 'Threshold', 0.3);
6
7  figure; imshow(img); title("Final detections (after post-processing)");
8  if ~isempty(bboxes)
9      annotation = string(labels) + " : " + string(round(scores,2));
10     imgOut = insertObjectAnnotation(img, "rectangle", bboxes, annotation);
11     imshow(imgOut);
12 end
```





```
1  from ultralytics import YOLO
2
3  model = YOLO("yolov8n.pt") # small, fast COCO model
4  results = model("highway.png", conf=0.3)
5
6  # bboxes, scores, labels like MATLAB
7  bboxes = results[0].boxes.xyxy.cpu().numpy() # [x1, y1, x2, y2]
8  scores = results[0].boxes.conf.cpu().numpy()
9  labels = results[0].boxes.cls.cpu().numpy().astype(int)
10
11 names = results[0].names
12 label_names = [names[i] for i in labels]
13
14 # annotated image
15 annotated = results[0].plot() # numpy image with boxes drawn
16
```

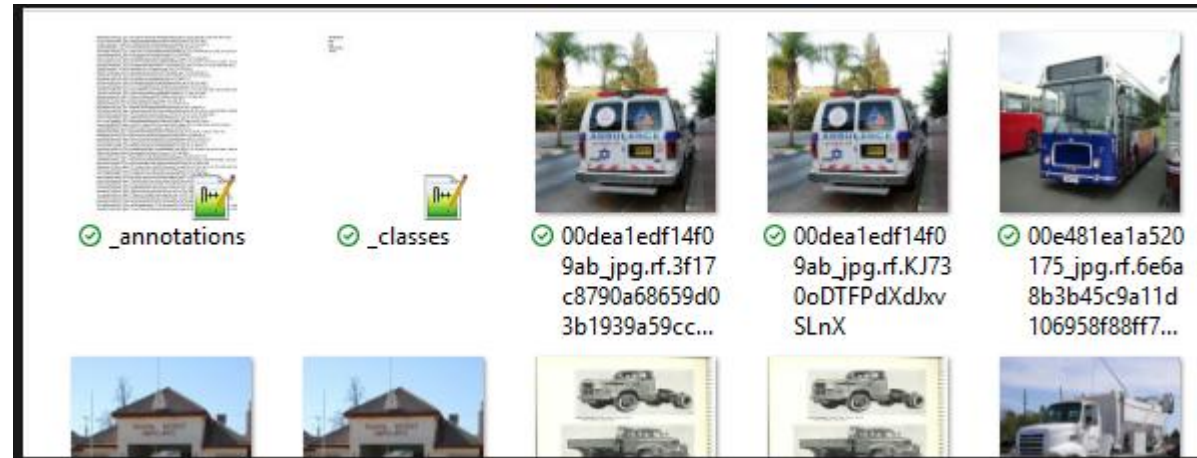


- **Data Preparation:**

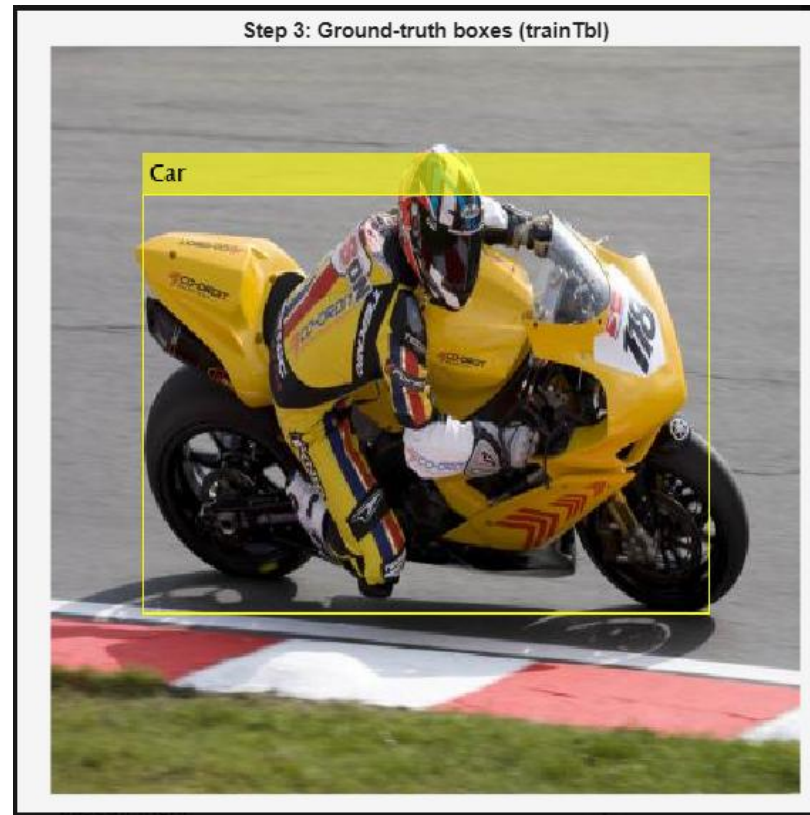
- Collect your images in one folder
- Label your images using **Image Labeler**
- Export a **groundTruth** object (or YOLO txt if you used a YOLO-format dataset)
- Note: public datasets (Kaggle etc.) can accelerate setup, but still require checks



```
0 0.5012019230769231 0.5408653846153846 0.7536057692307693
0.497596153846153850 0.4519230769230769 0.3485576923076923
0.5252403846153846 0.20072115384615385
```



- Label Quality Check:
 - Some dataset labels are incorrect/misaligned
 - Wrong labels → wrong learning signal → poor detection
 - Use visualization scripts to quickly detect annotation errors



- Step 2: Convert Labels to MATLAB Tables
 - Once images are labeled, transform image annotations into a structured table format
 - Formatting: YOLO labels are converted into MATLAB tables like *trainTbl* and *valTbl*.
 - Coordinate System: The boxes must be defined in pixel coordinates as [x, y, w, h]
- Step 3: Sanity Check
 - Use visualization scripts to overlay the Ground Truth (GT) boxes onto random 3
 - helps identify if labels are misaligned
- Step 4: Build Datastores
 - MATLAB's trainYOLOv4ObjectDetector does not take raw tables
 - Requires Datastores to manage memory and data flow.
 - Manages the loading of images, ensuring high-resolution files don't overwhelm your RAM



- Step 5 : Configuring the YOLOv4 Detector
 - Initializing the Network Architecture for object detection
- **Class Names:**
 - The specific categories the model must learn (e.g., 5 vehicle classes)
- **Anchor Boxes:**
 - Pre-defined bounding box shapes that represent the typical size distribution of your objects
 - If data has large trucks, anchors must include large shapes
 - If data has small motorcycles, need smaller anchor shapes
- **Input Size:**
 - Set to 416X416X3 to match the standard dataset resolution
 - 416X416 represents the height and width in pixels
 - Number 3 refers to the color channels, typically RGB (Red, Green, Blue)



- Step 6 — Train YOLOv4
 - Function: `trainYOLov4ObjectDetector(dsTrain, detector, options)`
 - Epoch: which pass over the whole training set you're on.
 - Iteration: how many mini-batches have been processed so far.
 - TimeElapsed: training time since start. LearnRate: current learning rate (can change if you use schedules).
 - TrainingLoss: loss on the current training mini-batches (should generally trend down, but it will fluctuate).
 - ValidationLoss: loss on validation set (computed every ValidationFrequency; helps detect overfitting).



- Step 6 — Train YOLOv4
- What is an Epoch?
 - An **Epoch** is one complete pass of the entire training dataset through the neural network
 - Eg: If dataset has 1,000 images, one epoch means the model has "seen" and tried to learn from every single one of those 1,000 images exactly once
- Underfitting: Setting MaxEpochs too low (e.g., 5)
 - Usually only for "smoke testing" to ensure the code runs.
 - The model won't have time to learn complex features.
- Overfitting: Setting it too high (e.g., 500)
 - Usually causes the model to "memorize" the training images,
 - losing its ability to generalize to new, unseen data.



- Step 6 — Train YOLOv4
- MiniBatchSize
 - Managing hardware constraints and the "smoothness" of learning
 - Determines how many images the model "sees" before it updates its internal weights
 - **MiniBatchSize is 2** means the model looks at 2 images,
 - calculates the error (loss), and then immediately tweaks its settings to get better
- Hardware (VRAM) Bottlenecks:
 - Smaller Batch (2-4): Fits on normal CPUs/GPUs
 - but takes longer to reach a stable "average" direction for learning.
 - Larger Batch (16-64): Enterprise GPUs (like an A100)
 - Provides a very stable gradient but requires high-end



- Step 6 — Train YOLOv4
- **What is TrainingLoss?**
 - **Loss = “how wrong the model is”** during training.
 - In YOLO, model is trying to learn **three things at the same time** for many positions in the image:
 1. **Where is the box?** (x, y, w, h)
 2. **Is there an object here?** (objectness score)
 3. **Which class is it?** (car/bus/truck etc.)
- YOLO's *total loss* is a combination of these parts:
 - **Box loss:** predicted box vs true box (shape + location)
 - **Objectness loss:** should this anchor/grid cell predict an object or not
 - **Classification loss:** correct class label or not
- **TrainingLoss specifically:**
 - It's the loss computed on the **training images** that the model is learning from.
 - High TrainingLoss = model is very wrong
 - Lower TrainingLoss = model is improving
 - It will usually **drop a lot early**, then drop slowly later.



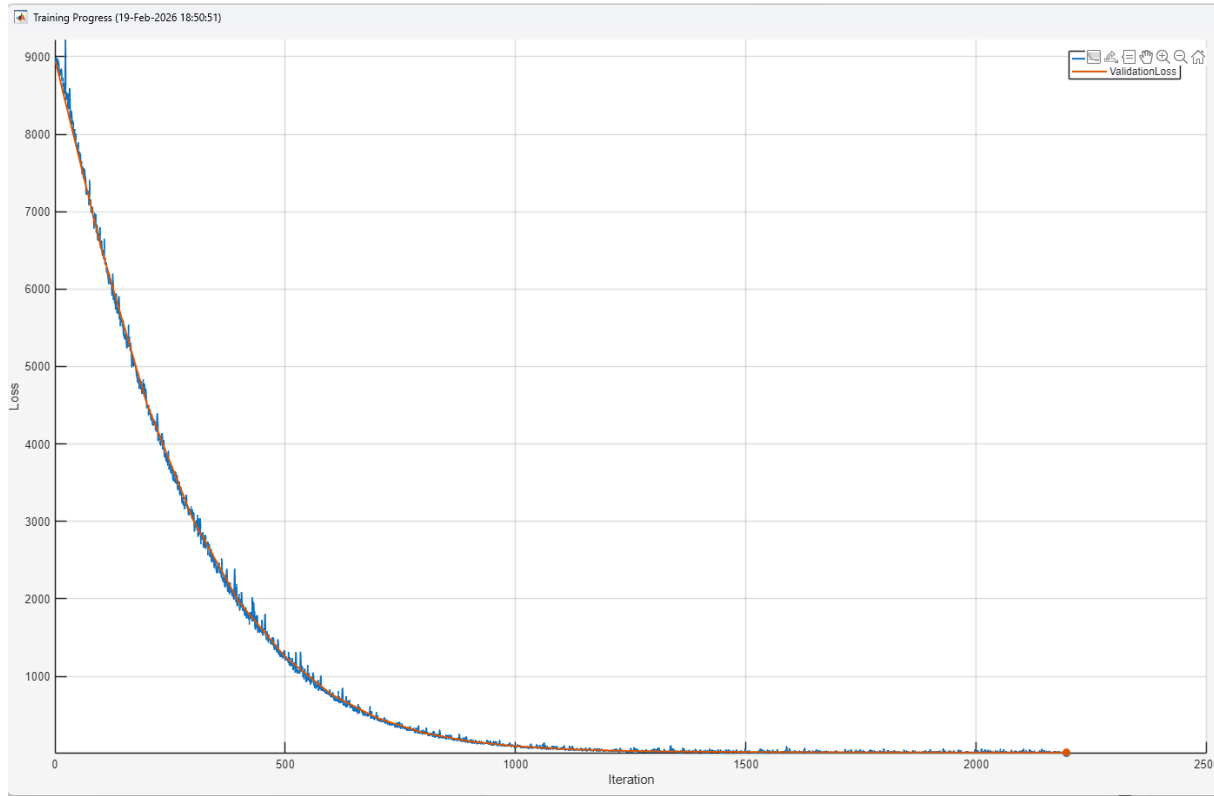
- Step 6 — Train YOLOv4
- **What is ValidationLoss?**
 - **Training Set:** The images the model "studies" from. It adjusts its weights based on these
 - **Training Loss** measures how well the model knows its "textbook."
 - **Validation Set:** Test on images the model **never** sees during training
 - **Validation Loss** measures how well the model can apply that knowledge to "new questions."
- True Learning
 - When both Training Loss and Validation Loss are moving downward together
 - Model is finding "General Patterns."
 - It realizes that a "car" usually has circular wheels, a windshield, and a certain height-to-width ratio



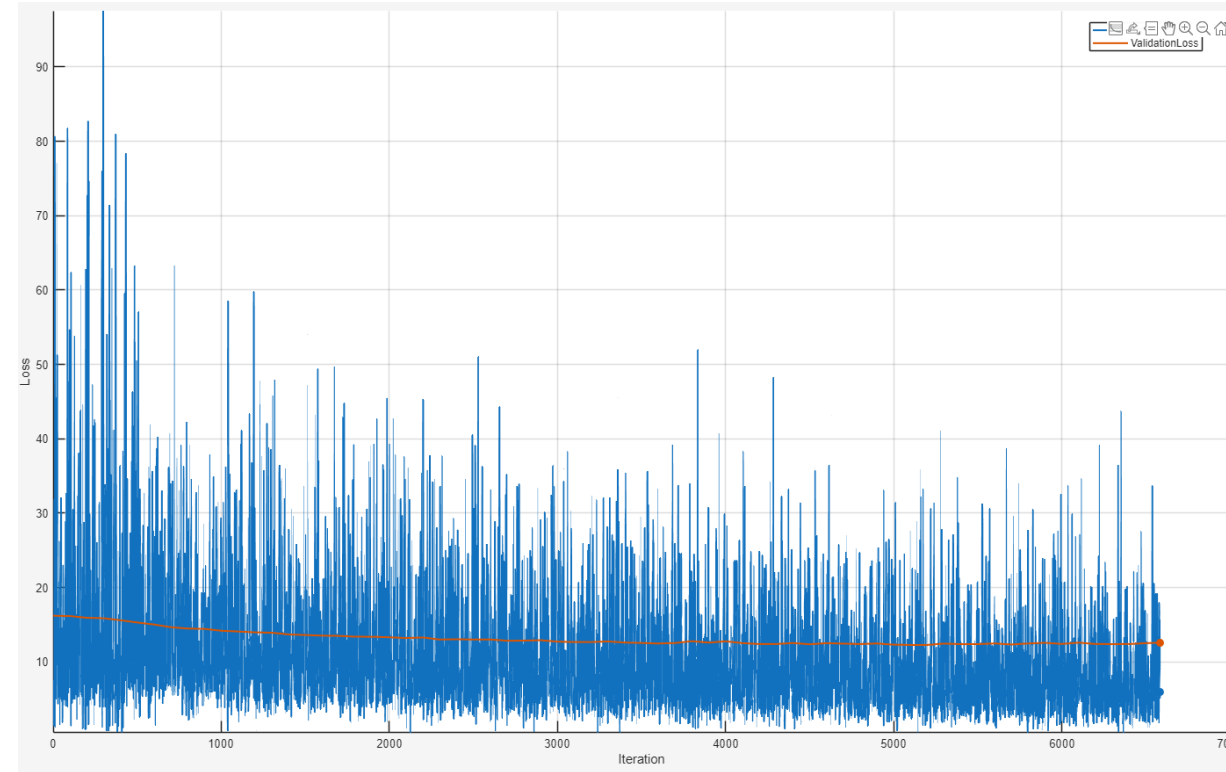
- Step 6 — Train YOLOv4
- Overfitting
 - Training Loss continues to drop but Validation Loss starts to flatten out or starts to climb back
 - Model learned the general patterns, so memorizing "noise" or specific patterns in training images
 - *Example:* It might learn that "Object #42 has a specific scratch on the bumper, so that's a car."

Training Loss	Validation Loss	Conclusion
Decreasing	Decreasing	Healthy Learning: The model is generalizing well.
Decreasing	Increasing	Overfitting: The model is memorizing; stop training.
Flat/High	Flat/High	Underfitting: The model is too simple or needs more epochs.
Increasing	Increasing	Divergence: Something is wrong

Training Loss Vs Validation Loss

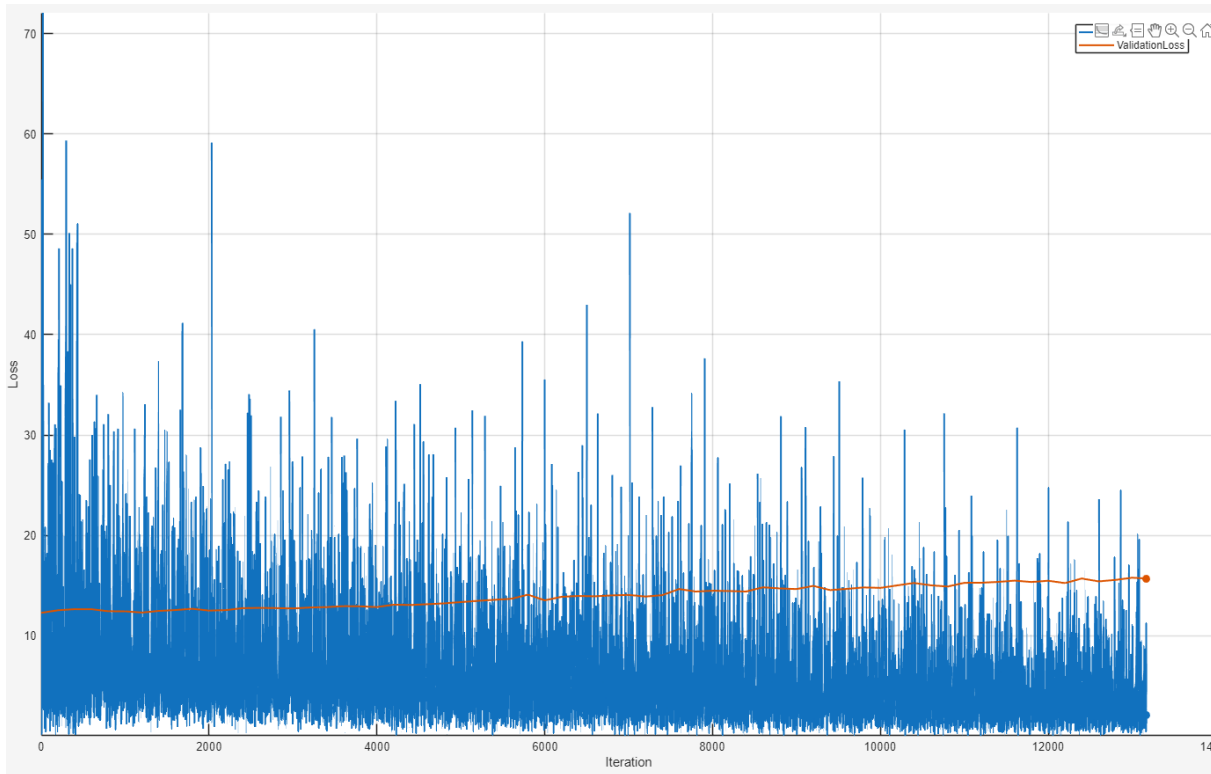


Epoch=5

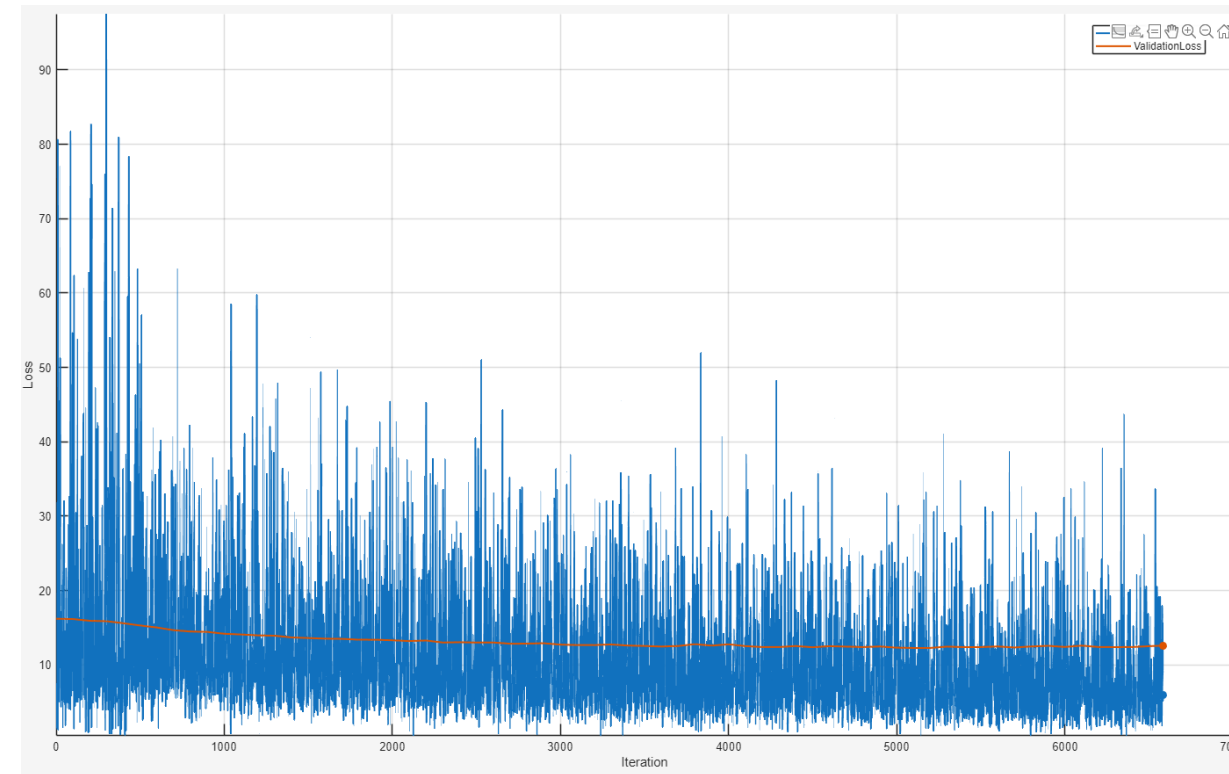


Epoch=15

Training Loss Vs Validation Loss



Epoch=30



Epoch=15

- Step 7 — Post-Training Data
- Trained Model (Detector):
 - Typically saved as yolo_detector_trained.mat
 - Contains **trained neural network weights**, pre-defined **anchor boxes**, and **class names**
 - Includes necessary post-processing logic, such as **Non-Maximum Suppression (NMS)**
- Training History
 - Data log used to evaluate the quality of the training
 - Contains the recorded **Training Loss** and **Validation Loss** values across all iterations
 - Includes the **learning rate schedule**, **timing** data, and total **iterations** completed
 - Essential for **analysis and reporting**, as well as **comparing different training runs**



- Step 8 — Test on new images

